AI APP DEVELOPMENT HANDBOOK

A Complete Spec-Driven Guide for Building AI-Assisted Web, Mobile, and Desktop Applications

---

Stack: Cursor · Supabase · Vercel · Claude

Platforms: Web · iOS · Android · Desktop

Anders Frederik Binau

2026

Table of Contents

1. Introduction

1.1 Purpose of This Guide
1.2 The Spec-Driven Workflow
1.3 User vs. AI Responsibilities
1.4 Tech Stack Overview
1B AI Product Interview Prompt

2. Repository Structure & AI Spec System

2.1 Master Folder Layout
2.2 AI Brain File (brain.md)
2.3 Product Spec (product.md)
2.4 Architecture File (architecture.md)
2.5 Database Spec (database.md)
2.6 Remaining Spec Files (ui.md, api.md, rules.md, debug.md, lessons.md, checklist.md)
2.6b Server-Side Supabase Client (/lib/supabase-server.ts)
2.6c Auth Middleware (/middleware.ts)
2.6d TypeScript Database Types
2.7 Input Validation Standard — Zod
2.8 Environment Variable Validation (/lib/env.ts)

3. Security: Supabase RLS System

3.1 Security Spec File (security.md)
3.2 Standard RLS Policy Patterns
3.3 Multi-User / Team Ownership
3.4 Secure Storage Buckets

4. Context Anchoring System

4.1 What Context Anchoring Is
4.2 The Four Core Anchors
4.3 Feature-Specific Anchors
4.4 Anchor-Based Prompting Workflow

5. Database Migration System

5.1 The Core Rule
5.2 Migration Folder & Naming
5.3 Safe Migration Pattern
5.4 Migration Planning File

5.5 Migration Checklist
5.6 Supabase Deployment Flow

6. Task System

6.1 Task File Structure
6.2 Example Task Files
6.3 Feature-by-Feature Workflow

7. Feature Packs (Reusable Modules)

7.1 What a Feature Pack Is
7.2 Feature Pack Library
7.3 Auth System Pack Example
7.4 Installing a Feature Pack
7.5 Building Your Library

8. Mega Prompt Library

8.1 Master Builder Prompt
8.2 Full Feature Builder
8.3 Database & Migration Prompts
8.4 UI & Page Prompts
8.5 Security, Performance & Audit Prompts
8.6 Feature Interpreter (Plain Text → Spec)
8.7 Debugging Prompts
8.8 Architecture Review & Refactor Prompts

9. Cross-Platform Development

9.1 Platform & Device Scope
9.2 AI Cross-Platform Pipeline Prompt
9.3 Device Layout System & Breakpoints
9.4 Device-Specific Edit Prompt
9.5 HTML Preview Generation
9.6 Multi-Device Preview
9.7 Platform Behavior Differences

10. Unified UI Schema (UUIS)

10.1 Core Concept
10.2 Design Tokens
10.3 Component Schema
10.4 Feature Schema
10.5 Platform Generation Prompt
10.6 Device Overrides
10.7 Preview & Diff Tooling

11. Full AI App Generator Architecture

11.1 Master Repo Structure
11.2 App Spec Layer
11.3 UI Schema Layer
11.4 Backend Schema Layer
11.5 Platform Generation Layer
11.6 Preview & Testing Layer
11.7 Full Pipeline Workflow

12. Site Analyzer Tool

12.1 Tool Goal & Pipeline
12.2 CLI Design
12.3 Node.js Scraper
12.4 AI Analysis Prompt
12.5 HTML Rebuild Prompt

13. AI Project Kickstart

13.1 Kickstart Prompt
13.2 Post-Kickstart Workflow

14. Best Practices & Advanced Tricks

14.1 Spec Locking
14.2 AI Memory (lessons.md)
14.3 Self-Auditing with Checklist
14.4 Edge Case & Security Review
14.5 Iterative Development Rhythm

15. Platform Migration System

15.1 What This System Covers
15.2 Migration Spec Folder Layout
15.3 Migration Brain File
15.4 AI Migration Interview Prompt
15.5 Gap Analysis
15.6 Risk Register
15.7 Backend Migration Task
15.8 Migration Execution Prompts
15.9 Zero-Downtime Cutover Pattern

16. Advanced Feature Builder

16.1 Scope of This Section
16.2 Advanced Feature Spec Folder
16.3 Advanced Feature Interview Prompt

16.4 Service Map Example
16.5 State Machine
16.6 Phased Implementation Plan
16.7 Advanced Feature Build Prompts

17. Outside Integrations Framework

17.1 Framework Overview
17.2 Integration Spec Folder
17.3 Integration Discovery Prompt
17.4 Integration Spec Example — Resend
17.5 Environment Variables
17.6 Setup Instructions
17.7 Integration Scaffold Prompt
17.8 Integration Test Checklist
17.9 Integration Index

18. User Data Migration System

18.1 What This System Covers
18.2 Migration Workspace Layout
18.3 Data Migration Interview Prompt
18.4 Schema Map Example
18.5 Transform Script
18.6 Validation Script
18.7 Import Script
18.8 User Account Migration — Auth Handling
18.9 Media Migration Script
18.10 Post-Migration Verification Prompts
18.11 HTML and Raw Text Source Handling

19. Quick-Start Implementation Reference

19 Phase 1 — Initialize Repository
19 Phase 2 — Define Spec Files
19 Phase 3 — Database Setup
19 Phase 4 — First Feature
19 Phase 5 — Security & Launch
19 Phase 6 — Add Feature Packs (Optional)

20. Summary & Next Steps

Appendix. Copyright

1 Introduction

Purpose, Workflow, Responsibilities, and Stack

1.1 Purpose of This Guide

This handbook consolidates a complete, production-grade system for building AI-assisted applications using a spec-driven architecture. It is designed for developers who want to use AI coding tools reliably across the full stack — from database schema to cross-platform UI — without the chaos that typically results from ad-hoc prompting.

The core thesis is simple: AI produces dramatically better code when it works from structured specifications rather than conversational guesses. This guide gives you the complete system — folder structure, spec files, prompts, migrations, context anchors, feature packs, and platform generation — in one place.

1.2 The Spec-Driven Workflow

The fundamental shift this system enforces is replacing the traditional *prompt → code* loop with a disciplined pipeline:

Step 1	**Interview / Define**	Clarify what the app does, who uses it, and what data it needs.
Step 2	**Spec Files**	Write product, architecture, database, UI, and security specs into /ai/.
Step 3	**Context Anchors**	Maintain short summaries the AI reads before every task.
Step 4	**Task Files**	Break every feature into a structured task spec in /ai/tasks/.
Step 5	**Migration**	Generate SQL migration files before touching any UI code.

Step 6	**Feature Build**	Prompt AI to implement the task following brain.md and the anchors.
Step 7	**Audit**	Run checklist, security review, and performance pass.
Step 8	**Iterate**	Update specs and anchors as the system evolves.

1.3 User vs. AI Responsibilities

Understanding the division of labor is essential. You own the architecture decisions; AI owns the implementation mechanics.

■ YOU (User)	■ AI (Claude / Cursor)
• Define the product idea and core features	• Conduct structured product interview
• Run the AI interview prompt to generate specs	• Generate all /ai spec files from interview
• Review and approve generated spec files	• Generate database migration SQL with RLS
• Create and maintain task files in /ai/tasks/	• Implement UI components from specs
• Apply SQL migrations to Supabase	• Wire Supabase queries to UI
• Review AI-generated code before merging	• Refactor and optimize existing code
• Update spec files when requirements change	• Perform security and performance audits
• Push to GitHub and deploy via Vercel	• Generate cross-platform UI from schema
• Maintain the lessons.md file	• Generate HTML previews for all devices

1.4 Tech Stack Overview

IDE	Cursor	AI-assisted code editing, file modification, Supabase wiring
AI Model	Claude (Anthropic)	Architecture, schema design, security review, complex logic
Backend	Supabase	PostgreSQL database, Auth, Row Level Security, Storage, Edge Functions
Frontend	Next.js	React-based web app, SSR/SSG, API routes
Hosting	Vercel	Automatic deployments, environment variables, edge network
Version Control	GitHub	Source of truth, CI/CD trigger for Vercel

■ Supabase Key Name Transition

Supabase is transitioning from the legacy NEXT_PUBLIC_SUPABASE_ANON_KEY to a new publishable key format (NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, value format: sb_publishable_xxx). Both formats are valid during the transition period and are interchangeable in all code in this guide. If your Supabase dashboard shows NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, use that value anywhere this guide shows NEXT_PUBLIC_SUPABASE_ANON_KEY. The variable name in your .env.local can be either — just ensure it matches what you reference in /lib/env.ts.

✓ Claude vs Cursor

Use Claude for architecture decisions, database schema design, security review, and any complex reasoning. Use Cursor for day-to-day file editing, UI wiring, and refactoring. Both read the same /ai spec files.

1B AI Product Interview Prompt

Let AI interview you and generate all spec files — the correct way to start any project

The Interview Approach

Instead of guessing your specs, use this prompt to have Claude act as a technical product manager and architect. It interviews you section by section, summarizes your answers, and then generates all spec files in one shot.

★ spec → task → code

This is the correct sequence. Never go directly from idea to code. The interview prompt enforces the right order: interview → architecture → tasks → code. This single habit is what separates AI toy projects from production-quality apps.

PROMPT — Full Product Interview Prompt — Part 1: Setup & Instructions

```
You are a senior software architect and product manager.

Your job is to help me fully design a web application
before any code is written. We will do this as an INTERVIEW.

Your responsibilities:
  1. Ask me structured questions about my application.
  2. Ask ONE section at a time.
  3. Do NOT skip important architecture questions.
  4. After each section, summarize my answers.
  5. Continue until the architecture is complete.

Once all questions are complete, generate:
  /ai/brain.md /ai/product.md /ai/architecture.md
  /ai/database.md /ai/ui.md /ai/api.md
  /ai/rules.md /ai/debug.md
  /ai/tasks/core_features.md
  /db/schema.sql
```

```

Default stack (override if I specify):
  Frontend: Next.js | Backend: Supabase
  Hosting: Vercel | Auth: Supabase Auth
```

PROMPT — Full Product Interview Prompt — Part 2: Interview Sections

```
SECTION 1: APPLICATION OVERVIEW
  - what the app does, who the users are,
    main value, core workflows

SECTION 2: USER TYPES
  - roles (admin, user, guest, etc.)
  - permissions, data ownership, moderation

SECTION 3: CORE FEATURES
  - main / optional / automation features
  - notifications, analytics, dashboards
  (Ask follow-ups if a feature is vague)

SECTION 4: DATA MODEL
  What data must exist? Relationships between entities?

SECTION 5: SECURITY
  - authentication, authorization
  - row-level permissions, admin controls, sensitive data

SECTION 6: UI STRUCTURE
  - main pages, dashboards, navigation, modals, forms

SECTION 7: API ACTIONS
  - create / edit / delete / search / filter / export / share

SECTION 8: AUTOMATIONS
  - background jobs, notifications, scheduled tasks, AI features

SECTION 9: SCALING
  - expected users, data size, real-time requirements

```

```
SECTION 10: INTEGRATIONS
  - payments / email / SMS / external APIs / file storage

CRITICAL RULES
  Ask clearly, one section at a time.
  Ask follow-up questions if answers are unclear.
  Do NOT generate files until interview is fully complete.
  Final output must be a production-ready spec repo.
```

Post-Interview Security Pass

After the interview generates your spec files, run this immediately before writing any code:

PROMPT — Edge Case & Security Review (Post-Interview)

```
Identify missing edge cases or security risks
in this architecture.

Look specifically for:
  - Privilege escalation (user accessing admin data)
  - Broken row-level security configurations
  - Missing indexes on frequently queried columns
  - Data leaks between users or organizations
  - Unprotected API actions
  - Missing ownership fields on tables

Report each issue with severity level and suggested fix.
```

2 Repository Structure & AI Spec System

The folder layout and spec files that anchor every AI interaction

2.1 Master Folder Layout

Every project begins with this structure. It separates application code, database artifacts, AI context files, and reusable feature packs into distinct, non-overlapping concerns.

■ **Folder Structure**

```
repo/
■■■ app/ # Next.js pages
■■■ components/ # UI components
■■■ lib/ # Utilities, Supabase clients, query functions
■ ■■■ env.ts # Environment variable validation
■ ■■■ supabase.ts # Browser client
■ ■■■ supabase-server.ts # SSR client (server components + actions)
■ ■■■ database.types.ts # Generated types (stub until first migration)
■ ■■■ schemas/ # Zod validation schemas
■■■ hooks/ # React custom hooks
■■■ styles/ # Global CSS
■■■ middleware.ts # Session refresh — required at root
|
■■■ db/
■ ■■■ schema.sql # Current full schema (always up to date)
■ ■■■ migrations/ # Numbered migration files
■ ■■■ 001_initial_schema.sql
■ ■■■ 002_add_characters.sql
■ ■■■ ...
|
■■■ ai/
■ ■■■ brain.md # Master AI instruction file
■ ■■■ product.md # What the app does
■ ■■■ architecture.md # Tech stack and data flow
■ ■■■ database.md # All tables and relationships
■ ■■■ ui.md # Pages and components
■ ■■■ api.md # All API functions/actions
■ ■■■ rules.md # Hard constraints for AI
■ ■■■ security.md # RLS rules and auth model
■ ■■■ debug.md # Debug procedure
■ ■■■ checklist.md # Feature completion checklist
■ ■■■ lessons.md # AI mistake log
■ ■■■ migrations.md # Database evolution plan
```

```
■ ■■■ migration_checklist.md
■ |
■ ■■■ context/ # Short anchor summaries
■ ■ ■■■ product_anchor.md
■ ■ ■■■ architecture_anchor.md
■ ■ ■■■ database_anchor.md
■ ■ ■■■ security_anchor.md
■ |
■ ■■■ tasks/ # One file per feature
■ ■■■ example_feature.md
|
■■■ feature-packs/ # Reusable plug-in modules
    ■■■ auth-system/
    ■■■ notifications/
    ■■■ teams/
    ■■■ admin-dashboard/
```

2.2 AI Brain File — /ai/brain.md

This is the single most important file. Every prompt to Cursor or Claude must start with *Follow /ai/brain.md*. It defines the development workflow, migration rules, architecture constraints, and forces the AI to read all other spec files before writing code.

■ **/ai/brain.md**

```
# AI PROJECT BRAIN

You are a senior software engineer working on this repository.

Before writing code you MUST read:
  /ai/product.md
  /ai/architecture.md
  /ai/database.md
  /ai/ui.md
  /ai/api.md
  /ai/rules.md
  /ai/security.md

Do not skip this step.

--------------------------------------------------
DEVELOPMENT RULES

1. Never modify the database schema without updating /db/schema.sql
2. All database access must use the Supabase client in /lib/supabase.ts (browser)
   or /lib/supabase-server.ts (server) — never create additional client instances
3. UI components belong in /components
4. Pages belong in /app
```

```
5. Prefer editing existing components instead of creating duplicates
6. Never introduce new frameworks or libraries unless explicitly requested
7. Do not modify unrelated files
8. Follow existing code style
9. Respect Supabase Row Level Security
10. Keep implementations simple
11. Server components and server actions must use /lib/supabase-server.ts, never
/lib/supabase.ts
12. middleware.ts handles session refresh — never remove, bypass, or exclude from the
matcher
13. Always use the Database generic type from /lib/database.types.ts with all
Supabase clients
14. Regenerate /lib/database.types.ts after every migration before writing new
queries
15. All user inputs must be validated with Zod before any database operation
16. Validation schema files live in /lib/schemas/[entity].schema.ts

--------------------------------------------------
DATABASE MIGRATION RULES

Never modify existing tables directly.

All schema changes must be written as a new file in:
  /db/migrations/

Each migration must:
1. Be numbered sequentially
2. Contain SQL only
3. Include RLS policies
4. Avoid destructive changes

--------------------------------------------------
SECURITY REQUIREMENT

All Supabase tables must follow the rules in:
  /ai/security.md

Never create tables without RLS policies.

--------------------------------------------------
FEATURE IMPLEMENTATION WORKFLOW

1. Read the task spec in /ai/tasks/
2. Identify database tables involved
3. Identify affected UI components
4. Generate migration if required
5. Implement backend queries in /lib
6. Implement UI
7. Connect Supabase queries
```

```
8. Verify using /ai/checklist.md
```

2.3 Product Spec — /ai/product.md

Describe what your app does, who the users are, and the primary entities. Keep it concise — this is a summary, not a full requirements document.

■ **/ai/product.md**

```
# PRODUCT SPEC

This application is a campaign manager for tabletop RPGs.

Users can:
- Create and manage game worlds
- Add towns and locations to worlds
- Create and track characters
- Log party events and encounters
- Generate random encounters

Primary Entities:
  World
  Town
  Character
  Event

User Types:
  admin — full access
  user — owns their own worlds
```

2.4 Architecture File — /ai/architecture.md

■ **/ai/architecture.md**

```
# ARCHITECTURE

Frontend: Next.js (React, TypeScript)
Backend: Supabase (PostgreSQL, Auth, Storage, Edge Functions)
Hosting: Vercel
Database: PostgreSQL via Supabase

Data Flow:
  Client → Supabase JS Client → PostgreSQL

Auth:
  Supabase Auth (email/password + magic link)

Environment Variables:
  NEXT_PUBLIC_SUPABASE_URL
```

```
  NEXT_PUBLIC_SUPABASE_ANON_KEY
```

2.5 Database Spec — /ai/database.md

List every table and its fields. This prevents AI from inventing non-existent tables or columns. Update this file whenever you run a migration.

■ **/ai/database.md**

```
# DATABASE SPEC

Table: worlds
  id uuid primary key
  name text
  description text
  user_id uuid references auth.users(id)

Table: towns
  id uuid primary key
  name text
  population int
  world_id uuid references worlds(id)
  user_id uuid references auth.users(id)

Table: characters
  id uuid primary key
  name text
  class text
  level int
  world_id uuid references worlds(id)
  user_id uuid references auth.users(id)

All user-owned tables include:
  user_id uuid references auth.users(id)
```

2.6 Remaining Spec Files

/ai/ui.md	List all pages and UI components. Pages go in /app; components in /components.
/ai/api.md	List all API actions as function signatures: createCharacter(world_id, name, class), etc.
/ai/rules.md	Hard constraints: always use /lib/supabase.ts, never modify schema.sql directly, etc.

`/ai/debug.md`	Debug procedure: identify error source → check Supabase queries → verify props → fix minimal code.
`/ai/lessons.md`	Running log of AI mistakes. Add entries whenever AI does something wrong, so it doesn't repeat.
`/ai/checklist.md`	Feature completion checklist: table exists, RLS enabled, UI connected, error handling, loading states, auth enforced.

Rules File — /ai/rules.md

Hard constraints AI must never violate. Keeps the AI from going off-stack regardless of what it thinks is a better approach.

■ `/ai/rules.md`

```
# DEVELOPMENT RULES

## Supabase Client
- Always use /lib/supabase.ts for browser (client components)
- Always use /lib/supabase-server.ts for server components, actions, routes
- Never create additional client instances anywhere else

## Schema
- Never modify schema.sql directly — all changes via migrations
- Never use DROP COLUMN on production tables
- All migrations must be reviewed before applying

## Code Style
- TypeScript strict mode — no `any` types
- All inputs validated with Zod before database operations
- Components in /components — pages in /app — queries in /lib

## AI Constraints
- Do not modify files outside the scope of the current task
- Do not install new packages without explicit instruction
- Do not deviate from /ai/security.md under any circumstances
```

Debug File — /ai/debug.md

The debug procedure AI follows when something breaks. Prevents it from rewriting unrelated code as a fix.

■ `/ai/debug.md`

```
# DEBUG PROCEDURE
```

```
When an error occurs, follow this sequence:

1. IDENTIFY — Is the error in the UI, the Supabase query, or the database?
   - UI errors: check component props and state
   - Query errors: check Supabase response and error object
   - DB errors: check RLS policies and column constraints

2. ISOLATE — Reproduce the error in the smallest possible scope.

3. CHECK — Before fixing:
   - Does the table exist in /db/schema.sql?
   - Is RLS enabled? Does the policy allow this operation?
   - Is user_id being passed correctly?
   - Is the Supabase client correct for this context (browser vs server)?

4. FIX — Modify only the file causing the error.
   Do not refactor unrelated code.

5. VERIFY — After fixing:
   - Does the fix break any existing functionality?
   - Does it violate any rule in /ai/security.md?
   - Run /ai/checklist.md for the affected feature.
```

Supabase Client — /lib/supabase.ts

All Supabase queries must flow through a single client instance. Never create additional client instances in components or pages.

■ **/lib/supabase.ts**

```
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from './database.types'
import { env } from './env'

export const supabase = createBrowserClient<Database>(
  env.NEXT_PUBLIC_SUPABASE_URL,
  env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

This file is updated progressively across Sections 2.6d and 2.8. Use the final version in Section 2.8 — it includes the Database generic type and env var validation.

2.6b Server-Side Supabase Client

The browser client in /lib/supabase.ts is for use in client components ('use client') only. Next.js App Router server components, server actions, and route handlers require a separate SSR-aware client that reads and writes cookies correctly. Without this, server-side session access returns null and auth tokens do not refresh.

■ **Install**

```
npm install @supabase/ssr
```

■ **/lib/supabase-server.ts**

```
// /lib/supabase-server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
import type { Database } from './database.types'
import { env } from './env'

export async function createSupabaseServerClient() {
  const cookieStore = await cookies()

  return createServerClient<Database>(
    env.NEXT_PUBLIC_SUPABASE_URL,
    env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // Server Components cannot set cookies directly.
            // The middleware.ts session refresh handles this.
          }
        },
      },
    }
  )
}
```

Client component ('use client')	`/lib/supabase.ts`
Server component (async function Page())	`/lib/supabase-server.ts`
Server action ('use server')	`/lib/supabase-server.ts`
Route handler (/app/api/*/route.ts)	`/lib/supabase-server.ts`

2.6c Auth Middleware — /middleware.ts

Required at the project root. Intercepts every request and refreshes the Supabase auth session token before it reaches any server component or API route. Without it, sessions expire mid-use and users are silently logged out with no recoverable state.

■ **/middleware.ts**

```
// /middleware.ts
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'
import { env } from './lib/env'

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient(
    env.NEXT_PUBLIC_SUPABASE_URL,
    env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll()
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          )
          supabaseResponse = NextResponse.next({ request })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  // IMPORTANT: Do not remove this line.
  // It refreshes the session and must run on every request.
  await supabase.auth.getUser()

  return supabaseResponse
}

export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico|.*\.(?:svg|png|jpg|jpeg|gif|webp)$).*
)',
  ],
}
```

■ **Do Not Remove getUser()**

> await supabase.auth.getUser() is what triggers the token refresh. The result is intentionally unused — its side effect on the cookie store is the purpose. Never remove this call or exclude middleware.ts from the matcher pattern.

> **■ Performance Note — getClaims() vs getUser()**
>
> getUser() makes a network request to the Supabase auth server on every call to verify the token — this is what makes it secure, but it does add latency on every request. Supabase introduced supabase.auth.getClaims() as a faster alternative that verifies the JWT locally without a network call. It is appropriate for non-security-critical middleware uses where you only need to read the user ID or role from the token, not confirm it against the auth server. For protecting pages and verifying identity before serving sensitive data, keep using getUser(). For reading the user ID to pass to a UI component where RLS will enforce security on the actual DB call, getClaims() may reduce middleware latency. Refer to the Supabase JS reference docs for the current getClaims() API.

2.6d TypeScript Database Types

Supabase generates a TypeScript type file directly from your live database schema. This gives every client call full type safety, IDE autocomplete, and compile-time error detection for table names, column names, and return shapes.

■ Regenerate Types

```
# Run after every migration — replace YOUR_PROJECT_ID with the ID from
# Supabase Dashboard → Project Settings → General
supabase gen types typescript --project-id YOUR_PROJECT_ID > lib/database.types.ts
```

Update /lib/supabase.ts to use the generated types:

■ /lib/supabase.ts (with Database generic)

```
// /lib/supabase.ts (updated with type safety)
import { createClient } from '@supabase/supabase-js'
import type { Database } from './database.types'

export const supabase = createClient<Database>(
  process.env.NEXT_PUBLIC_SUPABASE_URL!,
  process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)
```

> **■ Do not copy this version**
>
> It uses an outdated import library and unvalidated env vars. Use the final version in Section 2.8.

Initial placeholder (before first migration runs):

■ /lib/database.types.ts (stub)

```
// /lib/database.types.ts (stub — replace after first migration)
export type Database = {
  public: {
    Tables: Record<string, never>
    Views: Record<string, never>
    Functions: Record<string, never>
  }
}
```

2.7 Input Validation Standard — Zod

All user input must be validated before it reaches any Supabase database operation. This applies to server actions, API routes, and Edge Functions. No insert, update, or upsert should execute against unvalidated data.

■ **Install**

```
npm install zod
```

Schema convention: one file per entity in /lib/schemas/:

■ **/lib/schemas/character.schema.ts**

```
// /lib/schemas/character.schema.ts
import { z } from 'zod'

export const CreateCharacterSchema = z.object({
  name: z.string().min(1, 'Name is required').max(100),
  class: z.string().min(1, 'Class is required').max(50),
  level: z.number().int().min(1).max(20),
  world_id: z.string().uuid('Invalid world ID'),
})

export const UpdateCharacterSchema = CreateCharacterSchema.partial().extend({
  id: z.string().uuid(),
})

export type CreateCharacterInput = z.infer<typeof CreateCharacterSchema>
export type UpdateCharacterInput = z.infer<typeof UpdateCharacterSchema>
```

Validate in server actions or API routes before any database call:

■ **Usage in server action**

```
import { CreateCharacterSchema } from '@/lib/schemas/character.schema'

export async function createCharacter(formData: unknown) {
  const result = CreateCharacterSchema.safeParse(formData)

  if (!result.success) {
    return { error: result.error.flatten().fieldErrors }
  }

  // result.data is fully typed and safe to insert
```

```
  const { data } = await supabase
    .from('characters')
    .insert(result.data)

  return { data }
}
```

2.8 Environment Variable Validation — /lib/env.ts

If Supabase env vars are missing or misconfigured the app fails at runtime with a cryptic error. Since Zod is already in the stack, a build-time check costs nothing and catches the most common setup mistake immediately.

> **■ Supabase Key Name Transition**
>
> Supabase is transitioning from the legacy NEXT_PUBLIC_SUPABASE_ANON_KEY to a new publishable key format (NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, value format: sb_publishable_xxx). Both formats are valid during the transition period and are interchangeable in all code in this guide. If your Supabase dashboard shows NEXT_PUBLIC_SUPABASE_PUBLISHABLE_KEY, use that value anywhere this guide shows NEXT_PUBLIC_SUPABASE_ANON_KEY. The variable name in your .env.local can be either — just ensure it matches what you reference in /lib/env.ts.

■ /lib/env.ts

```
// /lib/env.ts
// Validates required environment variables at build time.
// Import this once from /lib/supabase.ts and /lib/supabase-server.ts.

import { z } from 'zod'

const envSchema = z.object({
  NEXT_PUBLIC_SUPABASE_URL: z.string().url('NEXT_PUBLIC_SUPABASE_URL must be a valid
URL'),
  NEXT_PUBLIC_SUPABASE_ANON_KEY: z.string().min(1, 'NEXT_PUBLIC_SUPABASE_ANON_KEY is
required'),
})

const parsed = envSchema.safeParse({
  NEXT_PUBLIC_SUPABASE_URL: process.env.NEXT_PUBLIC_SUPABASE_URL,
  NEXT_PUBLIC_SUPABASE_ANON_KEY: process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
})

if (!parsed.success) {
  console.error('■ Missing or invalid environment variables:')
  console.error(parsed.error.flatten().fieldErrors)
```

```
  throw new Error('Environment variable validation failed. Check .env.local.')
}

export const env = parsed.data
```

Update /lib/supabase.ts to import from env:

- **/lib/supabase.ts (with env validation)**

```
// /lib/supabase.ts
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from './database.types'
import { env } from './env'

export const supabase = createBrowserClient<Database>(
  env.NEXT_PUBLIC_SUPABASE_URL,
  env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

3 Security: Supabase RLS System

Row Level Security policies, ownership patterns, and secure storage

3.1 Security Spec File — /ai/security.md

This file is the source of truth for all database permissions. AI must read it before writing any SQL. It stops AI from generating dangerous open policies.

■ **/ai/security.md**

```
# SUPABASE SECURITY MODEL

Authentication is handled using Supabase Auth.
Users are identified by: auth.uid()

All user-owned data must include:
  user_id uuid references auth.users(id)

--------------------------------------------------
GENERAL SECURITY RULES

1. Row Level Security must be enabled on ALL tables.
2. Users may only access rows where: user_id = auth.uid()
3. Admin users may access all rows.
4. Never allow anonymous public read access unless explicitly required.
5. Insert operations must verify ownership.
6. Updates must only modify rows owned by the user.

--------------------------------------------------
CRITICAL RULE

Never create policies that allow:
  TRUE
or
  auth.role() = 'anon'
unless explicitly instructed.
```

■ **Recursion Warning — Admin Policies on profiles**

The admin override pattern queries profiles to check the user's role. If you apply this policy directly to the profiles table itself, PostgreSQL will throw an infinite recursion error (code 42P17) because evaluating the policy requires querying profiles, which triggers the policy again. Always use a SECURITY DEFINER helper function instead of an inline subquery. Add this function once in your initial schema migration, then reference it in all admin policies:

is_admin() helper function

```
-- Add this once — typically in your 001_initial_schema.sql migration
create or replace function public.is_admin()
returns boolean as $$
  select role = 'admin'
  from public.profiles
  where id = auth.uid();
$$ language sql security definer stable;
```

Admin policy using helper

```
-- Then use public.is_admin() in every admin policy:
create policy "Admins access all characters"
on characters for all
using ( public.is_admin() );
```

This function executes with the creator's privileges, bypassing RLS on profiles when checking the role — breaking the recursion.

3.2 Standard RLS Policy Patterns

Every table follows the same four-policy pattern. Shown here for the characters table as a canonical example.

Standard RLS Pattern

```
-- Table definition
create table characters (
  id uuid primary key default gen_random_uuid(),
  name text,
  class text,
  level int,
  user_id uuid references auth.users(id)
);

-- Enable RLS
alter table characters enable row level security;

-- SELECT: users read their own rows
create policy "Users read their characters"
  on characters for select
  using (user_id = auth.uid());

-- INSERT: user_id must match caller
```

```
create policy "Users insert their characters"
  on characters for insert
  with check (user_id = auth.uid());

-- UPDATE: users modify only their rows
create policy "Users update their characters"
  on characters for update
  using (user_id = auth.uid());

-- DELETE: users remove only their rows
create policy "Users delete their characters"
  on characters for delete
  using (user_id = auth.uid());

-- Admin override (if applicable)
-- Uses a SECURITY DEFINER function to avoid infinite recursion
-- when this pattern is applied to the profiles table itself.
create policy "Admins access all characters"
  on characters for all
  using ( public.is_admin() );
```

3.3 Multi-User / Team Ownership

When data is shared across an organization or team, use organization_id instead of user_id, and gate access via the memberships table.

■ Team RLS Pattern

```
-- Team-scoped SELECT policy
create policy "Members read org data"
  on projects for select
  using (
    organization_id IN (
      SELECT organization_id
      FROM memberships
      WHERE user_id = auth.uid()
    )
  );
```

3.4 Secure Storage Buckets

> **■ AI Forgets Storage Security**
>
> Supabase Storage buckets need their own RLS policies. AI will not add these unless explicitly told. Include storage policies in your security spec.

■ Storage RLS

```
-- Storage upload policy
create policy "Users upload their files"
  on storage.objects for insert
  with check (
    bucket_id = 'user-files'
    AND auth.uid()::text = (storage.foldername(name))[1]
  );

-- Storage read policy
create policy "Users read their files"
  on storage.objects for select
  using (
    bucket_id = 'user-files'
    AND auth.uid()::text = (storage.foldername(name))[1]
  );
```

4 Context Anchoring System

Keeping AI aligned across large repos with short, always-read summaries

4.1 What Context Anchoring Is

Once a repository grows beyond 30–50 files, LLMs begin to forget architecture details, invent duplicate components, and break existing patterns. Context Anchors solve this by providing short, focused summaries of each critical spec area that the AI reads at the start of every task.

The key insight: a 500-line database spec gets forgotten mid-task. A 20-line anchor summary stays resident throughout. Anchors do not replace the full spec files — they complement them.

4.2 The Four Core Anchors

■ Why Anchors Work

Large spec files like database.md or architecture.md may be 500–1000 lines. LLMs prefer short context — they stay more consistent when critical information is condensed into a 15–20 line summary they can hold in working memory throughout a task. Anchors are those summaries.

/ai/context/product_anchor.md

3–10 bullet points: app name, core user actions, primary entities. Nothing more.

■ /ai/context/product_anchor.md

```
# PRODUCT ANCHOR

App: Campaign Manager (tabletop RPG tool)

Users can:
- Create game worlds
- Add towns and locations
- Create characters
- Log party events

Primary entities: worlds, towns, characters, events
```

/ai/context/architecture_anchor.md

Stack, auth, and client-DB data flow in under 15 lines.

- /ai/context/architecture_anchor.md

```
# ARCHITECTURE ANCHOR

Stack: Next.js + Supabase + Vercel

Auth: Supabase Auth (auth.uid() identifies users)

Client communicates with DB using:
  /lib/supabase.ts — browser client (use client components)
  /lib/supabase-server.ts — server client (server components, actions, routes)
```

/ai/context/database_anchor.md

Core table list, key relationships, and ownership rule.

- /ai/context/database_anchor.md

```
# DATABASE ANCHOR

Core Tables:
  users, worlds, towns, characters, events

Relationships:
  worlds → towns (world_id)
  worlds → characters (world_id)
  towns → events (town_id)

All user-owned tables include:
  user_id uuid references auth.users(id)
```

/ai/context/security_anchor.md

RLS requirement summary. AI must check this before writing any SQL.

- /ai/context/security_anchor.md

```
# SECURITY ANCHOR

RLS enabled on ALL tables.

Users access only rows where:
  user_id = auth.uid()

Admins access all rows.
Never use: USING (TRUE) or auth.role() = 'anon'
```

4.3 Feature-Specific Anchors

For large subsystems, create dedicated feature anchors. Reference them in prompts when working on that subsystem.

- **/ai/context/character_system_anchor.md**

```
# CHARACTER SYSTEM ANCHOR

Tables: characters, inventory
Components: CharacterCard, CharacterModal, InventoryList
Queries: /lib/characters.ts

Actions:
  createCharacter(world_id, name, class, level)
  deleteCharacter(id)
  updateCharacter(id, fields)
  getCharacters(world_id)
```

4.4 Anchor-Based Prompting Workflow

Start every non-trivial Cursor or Claude prompt with the anchor set. This is the single most effective habit for keeping AI consistent across sessions.

PROMPT — Standard Anchor Preamble

```
Read these files before making changes:

/ai/brain.md
/ai/context/product_anchor.md
/ai/context/architecture_anchor.md
/ai/context/database_anchor.md
/ai/context/security_anchor.md

Then implement:
/ai/tasks/[feature_name].md
```

5 Database Migration System

Safe, versioned schema evolution that AI must follow

5.1 The Core Rule

■ Never Modify the Database Directly

AI must never alter an existing table in-place or overwrite schema.sql directly. Every schema change — new table, new column, new policy — must be a new numbered migration file. This mirrors professional backend practice and prevents silent data loss.

5.2 Migration Folder & Naming Convention

■ Migration Folder

```
db/
■■■ migrations/
    ■■■ 001_initial_schema.sql
    ■■■ 002_create_characters.sql
    ■■■ 003_add_inventory_table.sql
    ■■■ 004_add_character_level_index.sql
    ■■■ 005_add_events_table.sql
```

Naming rule: ###_**description.sql** — three-digit number followed by a snake_case description. Never reuse or overwrite a number. Never delete old migrations.

■ 002_create_characters.sql

```
-- 002_create_characters.sql

create table characters (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  class text,
  level int default 1,
  world_id uuid references worlds(id),
  user_id uuid references auth.users(id)
);

alter table characters enable row level security;
```

```
create policy "Users read characters"
  on characters for select
  using (user_id = auth.uid());

create policy "Users insert characters"
  on characters for insert
  with check (user_id = auth.uid());

create policy "Users update characters"
  on characters for update
  using (user_id = auth.uid());

create policy "Users delete characters"
  on characters for delete
  using (user_id = auth.uid());

-- Always index foreign keys
create index idx_characters_user
  on characters(user_id);

create index idx_characters_world
  on characters(world_id);
```

Every migration must include: table creation, RLS enablement, all four CRUD policies, and indexes on foreign key columns. AI often omits indexes unless explicitly reminded.

■ Example Migration File

```
-- 003_add_inventory_table.sql

create table inventory_items (
  id uuid primary key default gen_random_uuid(),
  name text not null,
  description text,
  quantity int default 1,
  character_id uuid references characters(id) on delete cascade,
  user_id uuid references auth.users(id)
);

alter table inventory_items enable row level security;

create policy "Users read their inventory"
  on inventory_items for select
  using (user_id = auth.uid());

create policy "Users insert inventory"
  on inventory_items for insert
  with check (user_id = auth.uid());

create policy "Users update inventory"
```

```
  on inventory_items for update
  using (user_id = auth.uid());

create policy "Users delete inventory"
  on inventory_items for delete
  using (user_id = auth.uid());

-- Index for performance
create index idx_inventory_character
  on inventory_items(character_id);

create index idx_inventory_user
  on inventory_items(user_id);
```

5.3 Safe Migration Pattern

Never use DROP COLUMN on production tables. Instead, add the new column and migrate data.

■ **Safe Column Change**

```
-- Add new column with default (safe)
alter table characters
  add column level int default 1;

-- After verifying data, deprecate old column in a later migration
-- Never use: alter table characters drop column old_field;
```

5.4 Migration Planning File — /ai/migrations.md

■ **/ai/migrations.md**

```
# DATABASE EVOLUTION PLAN

Version 1 (initial)
  - users (via Supabase Auth)
  - profiles
  - worlds

Version 2
  - towns
  - characters

Version 3
  - inventory_items

Version 4
  - events
  - quests
```

5.5 Migration Checklist — /ai/migration_checklist.md

■ /ai/migration_checklist.md

```
Before finalizing any migration, verify:

[ ] Table has a UUID primary key with default gen_random_uuid()
[ ] RLS enabled on the table
[ ] SELECT policy exists
[ ] INSERT policy with WITH CHECK exists
[ ] UPDATE policy exists
[ ] DELETE policy exists
[ ] Foreign key columns have indexes
[ ] No destructive DROP or TRUNCATE operations
[ ] schema.sql updated to reflect current state
[ ] Run supabase gen types typescript --project-id YOUR_PROJECT_ID >
lib/database.types.ts
[ ] Commit updated database.types.ts alongside the migration file
```

5.6 Supabase Deployment Flow

1. Write or generate migration file in /db/migrations/
2. Review migration — check for RLS, indexes, no destructive ops
3. Open Supabase SQL Editor and run the migration
4. Verify table and policies in Supabase Table Editor
5. Update /db/schema.sql to reflect current full schema
6. Commit migration file and updated schema.sql to git
7. Run supabase gen types typescript and commit the updated database.types.ts

■ Local Development with Supabase CLI (Recommended for Teams)

For any project with more than one developer, or where you want to test migrations before applying them live, use the Supabase CLI local environment:

■ Local Supabase CLI

```
# Start local Supabase (requires Docker)
supabase start

# Apply your migrations locally before pushing to remote
supabase db push --local

# Generate types from your local schema
supabase gen types typescript --local > lib/database.types.ts

# Stop local environment
```

```
supabase stop
```

- **.env.test (never .env.local or .env)**

```
TEST_SUPABASE_URL=http://127.0.0.1:54321
TEST_SUPABASE_SERVICE_ROLE_KEY=your-local-service-role-key
```

Add .env.test to .gitignore. Never run integration tests against your production Supabase project.

6 Task System

The most important workflow element — spec before code, always

6.1 Task File Structure

Every feature must have a task file in /ai/tasks/ before any code is written. This is the single most effective practice for getting reliable AI output. Without a task file, AI guesses. With one, it executes.

■ **/ai/tasks/[feature].md template**

```
# TASK: [Feature Name]

## GOAL
[One sentence describing what this feature does for the user.]

## DATABASE
Tables involved:
  - [table_name] — [brief description]

Fields required (if new):
  - field_name type [nullable/required]

## UI
Components required:
  - [ComponentName] — [what it does]

Pages affected:
  - /app/[page]/page.tsx

## VALIDATION (Zod)
  - Schema file: /lib/schemas/[entity].schema.ts
  - Fields to validate: [field list with types and constraints]

## BACKEND
Query functions required in /lib/[feature].ts:
  - [functionName(params)] → return type

## BEHAVIOR
Step-by-step user workflow:
1. User does X
2. System does Y
3. Result Z is shown
```

```
## API ACTIONS
Functions to implement:
  - actionName(params) → return type

## DONE WHEN
  - [ ] User can [primary action]
  - [ ] Data persists to database
  - [ ] Only the user's own data is visible
  - [ ] Error states handled

## NOTES
Any edge cases, permissions, or special constraints.
```

6.2 Example Task File

■ **/ai/tasks/create_character.md**

```
# TASK: Create Character

## GOAL
Allow users to create new characters belonging to their world.

## DATABASE
Tables:
  - characters — stores all character data

Fields:
  - name text required
  - class text required
  - level int default 1
  - world_id uuid required (FK to worlds)
  - user_id uuid required (FK to auth.users)

## UI
Components:
  - CharacterModal — form modal for character creation
  - CharacterCard — display card shown after creation

Pages:
  - /app/worlds/[id]/characters/page.tsx

## BEHAVIOR
1. User clicks "New Character" on character list page
2. CharacterModal opens with form fields
3. User fills in name, class, level
4. On submit, insert into characters table with user_id and world_id
5. Modal closes, character list refreshes
```

```
## API ACTIONS
  createCharacter(world_id, name, class, level) → character
  getCharacters(world_id) → character[]

## DONE WHEN
- [ ] User can create a new character from the character list page
- [ ] Character is saved to the database with correct user_id and world_id
- [ ] Only the current user's characters are visible in the list
- [ ] Form displays validation errors for missing name or class
- [ ] Modal closes and list refreshes after successful creation

## NOTES
  - user_id must be set server-side or via RLS check
  - Refresh list using React Query or Supabase realtime
```

6.3 Feature-by-Feature Workflow

The complete loop from idea to deployed feature:

1. **Define** — Write the task spec in /ai/tasks/[feature].md
2. **Migrate** — Prompt AI to generate the migration SQL, review it, apply to Supabase
3. **Schema** — Update /db/schema.sql to reflect current state
4. **Backend** — Prompt AI to generate Supabase queries in /lib/[feature].ts
5. **UI** — Prompt AI to build components and connect queries
6. **Audit** — Run checklist prompt to verify completeness
7. **Commit** — Commit migration, schema, lib, components, and pages together

7 Feature Packs (Reusable Modules)

Plug-and-play systems that eliminate rebuilding the same features across projects

7.1 What a Feature Pack Is

A Feature Pack is a self-contained folder holding everything needed to install a complete subsystem — auth, notifications, teams, file uploads, etc. — into any project. After building a few projects, your pack library eliminates weeks of repeated setup.

Folder Structure

```
feature-packs/
■■■ auth-system/
■ ■■■ feature.md # What this pack does
■ ■■■ schema.sql # Tables required
■ ■■■ policies.sql # RLS policies
■ ■■■ tasks.md # Implementation steps
■ ■■■ components/ # UI components
■ ■■■ LoginForm.tsx
■ ■■■ RegisterForm.tsx
■■■ notifications/
■■■ teams/
■■■ admin-dashboard/
■■■ file-uploads/
■■■ activity-logs/
■■■ search/
```

7.2 Recommended Feature Pack Library

`auth-system`	Email/password + magic link, profiles table, role system
`admin-dashboard`	User management, metrics view, activity log viewer
`notifications`	In-app notifications table, mark-read, notification bell UI
`teams`	Organizations + memberships, team-scoped RLS, invite system

`file-uploads`	Supabase Storage buckets, upload UI, signed URL retrieval
`activity-logs`	Audit trail for all user actions with timestamps
`search`	Full-text search UI with debouncing, Supabase text search queries
`analytics`	Page view tracking, event logging, simple metrics dashboard

7.3 Auth System Pack — Complete Example

■ `feature-packs/auth-system/feature.md`

```
# AUTH SYSTEM FEATURE PACK

Users can:
- Register with email + password
- Login and logout
- Reset password via email
- View and edit their profile

Tables:
  profiles

Authentication:
  Supabase Auth (auth.users is managed by Supabase)
```

■ `feature-packs/auth-system/schema.sql`

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text,
  name text,
  role text default 'user' check (role in ('admin', 'user', 'moderator')),
  avatar text,
  created_at timestamptz default now()
);

alter table profiles enable row level security;

create policy "Users read own profile"
  on profiles for select
  using (id = auth.uid());

create policy "Users update own profile"
  on profiles for update
  using (id = auth.uid());

-- Trigger: auto-create profile on signup
create or replace function public.handle_new_user()
```

```
returns trigger as $$
begin
  insert into public.profiles (id, email)
  values (new.id, new.email)
  on conflict (id) do nothing;
  return new;
exception when others then
  -- Log the error but allow user creation to proceed.
  -- A missing profile is recoverable; a failed signup is not.
  raise warning 'handle_new_user: profile insert failed for user %: %', new.id,
sqlerrm;
  return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

-- Helper function: used in all admin RLS policies.
-- SECURITY DEFINER prevents infinite recursion when applied to the profiles table.
create or replace function public.is_admin()
returns boolean as $$
  select role = 'admin'
  from public.profiles
  where id = auth.uid();
$$ language sql security definer stable;
```

■ Note

The profiles table intentionally omits INSERT and DELETE policies. The handle_new_user trigger (which runs as SECURITY DEFINER) creates profiles on signup, bypassing RLS. Direct INSERT by users is not permitted by design. If account deletion is required, add a DELETE policy scoped to the owner. The migration checklist FAIL on these two items is expected for this specific table.

7.4 Installing a Feature Pack

PROMPT — Install Feature Pack

```
Follow /ai/brain.md.

Integrate the feature pack at:
  /feature-packs/auth-system/

Tasks:
```

```
1. Review schema.sql — do not apply if table already exists
2. Generate migration from schema.sql as:
   /db/migrations/00X_auth_system.sql
3. Integrate UI components from components/ into /components
4. Connect Supabase queries using /lib/supabase.ts
5. Update /ai/database.md with the new profiles table
```

7.5 Building Your Library

Every feature pack in your library starts as a working feature in a real project. Once a feature is stable and audited, extract it into a pack. Future projects install it in minutes instead of hours.

When to Extract

Extract a feature into a pack when:

• It has been running in production without modification for at least one sprint

• It passes the full /ai/checklist.md verification

• It has no hard-coded references to the host project (no specific table names, routes, or IDs from the parent app)

Extraction Process

■ Extraction steps

```
1. Create the pack folder:
   /feature-packs/[feature-name]/

2. Copy the migration SQL into schema.sql — strip any host-project-specific
   foreign keys and replace with clear TODO comments where the installer
   must wire up their own relationships.

3. Separate the RLS policies into policies.sql if they are not already
   embedded in schema.sql.

4. Copy the /lib/[feature].ts query file into the pack folder.
   Replace hard-coded table references with documented assumptions.

5. Copy UI components into /feature-packs/[feature-name]/components/.
   Remove any imports that reference host-project-specific files.
   Replace with prop-driven interfaces.

6. Write feature.md — document what the pack does, what tables it creates,
   what components it includes, and what the installer must configure.
```

```
7. Write tasks.md — the installation steps a developer follows when
   running the Install Feature Pack prompt (Section 7.4).
```

Minimum Pack Structure

■ **Pack folder structure**

```
feature-packs/
■■■ [feature-name]/
    ■■■ feature.md # What it does, tables, components, install requirements
    ■■■ schema.sql # Table definitions with RLS and indexes
    ■■■ policies.sql # RLS policies (if separate from schema.sql)
    ■■■ tasks.md # Step-by-step installation instructions
    ■■■ components/ # UI components, prop-driven, no host-project imports
        ■■■ [Component].tsx
```

feature.md Template

■ **feature.md template**

```
# FEATURE PACK — [Name]

## What It Does
[One paragraph description]

## Tables Created
- [table_name] — [what it stores]

## Components Included
- [ComponentName] — [what it renders and what props it expects]

## Install Requirements
- Depends on: [other tables or packs this pack requires, e.g., profiles]
- Env vars required: [any new env vars]
- Manual steps: [anything the developer must do outside of SQL and code]

## Install Instructions
See tasks.md.
```

Keeping Packs Current

When you improve a feature in one project — a new RLS pattern, a refactored component, a better query — update the pack. Treat it like a shared library. A pack that diverges from current best practices loses its value.

8 Mega Prompt Library

Copy-paste prompts organized by purpose — always begin with Follow /ai/brain.md

8.1 Master Builder Prompt

■ Never Use the Service Role Key in Edge Functions

SUPABASE_SERVICE_ROLE_KEY bypasses Row Level Security entirely — any code running with this key has unrestricted read/write access to your entire database. It is only appropriate for server-side scripts that run in isolation (such as the data migration scripts in Section 18). Edge Functions are callable from the internet. If an Edge Function uses the service role key and contains any vulnerability (injection, logic flaw, missing auth check), your entire database is exposed. Use the anon key or a scoped service account in Edge Functions, and rely on RLS policies to enforce data access. Only use the service role key in offline migration scripts — and revoke it when the migration is complete.

Use this as the preamble for any complete feature build. It forces AI through a disciplined seven-step engineering process.

PROMPT — Master AI Builder Prompt

```
You are a senior full-stack engineer responsible for implementing
features in this repository.

Before writing any code you MUST read:
  /ai/brain.md
  /ai/security.md
  /ai/context/product_anchor.md
  /ai/context/architecture_anchor.md
  /ai/context/database_anchor.md

Follow all rules defined in those files.

DEVELOPMENT PROCESS

STEP 1 — Understand the feature
```

```
  Explain the feature and what system parts are affected.

STEP 2 — Database planning
  If a DB change is required, create a new migration in /db/migrations/.

STEP 2b — Validation schema
  Create Zod schema in /lib/schemas/[entity].schema.ts.
  Validate all user inputs before any database operation.

STEP 3 — Security planning
  Ensure all tables follow RLS rules in /ai/security.md.

STEP 4 — Backend implementation
  Create or update queries in /lib.

STEP 5 — UI implementation
  Create or modify components in /components.

STEP 6 — Page integration
  Update pages in /app.

STEP 7 — Validation
  Verify feature using /ai/checklist.md.

CRITICAL RULES
  Never modify unrelated files.
  Never change existing migrations.
  Always generate new migrations for schema changes.
  Prefer modifying existing components over creating duplicates.

OUTPUT FORMAT
  1. Feature plan
  2. Migration SQL (if needed)
  3. Backend queries
  4. UI components
  5. Integration steps
```

8.2 Full Feature Builder — Quick Build

Use when /ai/brain.md is already loaded in context and you don't need the full role/workflow preamble from Section 8.1. The Master Builder Prompt (Section 8.1) is the canonical full-feature prompt — this is a no-preamble quick-reference variant.

PROMPT — Quick Build (no preamble — use when brain.md is loaded in context)

```
Follow /ai/brain.md.

Feature description:
[paste feature description]

Required steps:
1. Plan feature architecture
2. Generate database migration if needed
2b. Create Zod validation schema in /lib/schemas/[entity].schema.ts
3. Implement RLS policies
4. Update schema.sql
5. Create backend queries in /lib
6. Create UI components
7. Integrate with pages in /app
8. Verify using /ai/checklist.md
```

8.3 Database & Migration Prompts

PROMPT — Database Table Generator

```
Follow:
  /ai/brain.md
  /ai/security.md
  /ai/database.md

Generate a new database migration for:

TABLE: [name]

FIELDS:
  [field_name] [type] [constraints]
```

```
RELATIONSHIPS:
  [table] belongs to [other table] via [foreign key]

Include:
  - RLS policies (all four: SELECT, INSERT, UPDATE, DELETE)
  - Indexes on all foreign key columns
  - UUID primary key with gen_random_uuid() default
  - Zod validation schema at /lib/schemas/[table].schema.ts
    with fields matching the table definition
```

PROMPT — Supabase Query Builder

```
Follow /ai/brain.md.

Generate Supabase query functions for table: [table_name]

Create the following in /lib/[table].ts:
  - create[Entity](fields) → entity
  - update[Entity](id, fields) → entity
  - delete[Entity](id) → void
  - list[Entities](filter?) → entity[]
  - search[Entities](query) → entity[]

Use the Supabase client from /lib/supabase.ts.
Include TypeScript types for all parameters and return values.
Import and apply the Zod schema from /lib/schemas/[entity].schema.ts
before any insert or update operation.
```

8.4 UI & Page Prompts

PROMPT — UI Component Generator

```
Follow /ai/brain.md.

Create UI components for: [feature]

Components required:
  [ComponentName] — [what it does and what data it receives]
```

```
Rules:
  - Place all components in /components/
  - Use existing styling patterns in the repository
  - Connect to Supabase queries from /lib/
  - Include loading and error states
  - Use TypeScript props interfaces
```

PROMPT — Page Generator

```
Follow /ai/brain.md.

Create a new page: /app/[route]/page.tsx

The page must:
  - List all [entity] records for the current user
  - Allow creating a new [entity]
  - Allow editing an existing [entity]
  - Allow deleting [entity] with confirmation

Use:
  - Supabase queries from /lib/[entity].ts
  - Existing UI component patterns
  - Loading skeleton while data fetches
  - Empty state when no records exist
```

8.5 Security, Performance & Audit Prompts

PROMPT — Security Audit (quick inline check)

```
Review the repository using /ai/security.md.

Identify and report:
  - Tables missing RLS enablement
  - Tables missing any of the four standard policies
  - Queries that bypass RLS
  - Potential data leaks between users
  - Storage buckets without policies
  - Any use of USING (TRUE) or anon access

```

```
Output a prioritized list of issues with suggested fixes.

For a full post-architecture security review, use the
Post-Architecture Security Review prompt in Section 14.4.
```

PROMPT — Performance Audit

```
Follow /ai/brain.md.

Analyze the repository for performance issues.

Identify:
  - Queries missing indexes on foreign keys
  - N+1 query patterns in components
  - Unnecessary React re-renders
  - Missing pagination on list queries
  - Unoptimized Supabase queries

Suggest specific fixes for each issue found.
```

PROMPT — Checklist Verification

```
Follow /ai/checklist.md.

Verify that the [feature name] feature implementation is complete.

For each checklist item, report:
  PASS — if complete
  FAIL — if missing or incomplete, with the specific gap

Do not mark anything PASS unless you can point to the exact code.
```

8.6 Feature Interpreter — Plain Text → Spec

This is one of the most powerful prompts in the system. Write a rough feature idea in plain English, and AI converts it into a fully structured implementation plan.

PROMPT — Master Feature Interpreter

```
You are a senior software architect.

```

```
Convert the following plain-language feature request into a full
structured implementation plan for this repository.

Before processing, read:
  /ai/brain.md
  /ai/security.md
  /ai/context/product_anchor.md
  /ai/context/database_anchor.md

PROCESS:

STEP 1: Interpret the feature and explain what it likely means.

STEP 2: Ask up to 5 clarifying questions.
  Allow the user to answer or type SKIP.

STEP 3: Design the feature architecture:
  - Database tables required
  - Relationships and foreign keys
  - UI components needed
  - API actions

STEP 4: Generate the task specification file:
  /ai/tasks/[feature_name].md

STEP 5: Determine if a migration is required.
  If yes, generate migration SQL.

STEP 6: Plan UI components.

STEP 7: Generate a step-by-step implementation plan.

OUTPUT:
  1. Feature explanation
  2. Clarifying questions (or SKIP)
  3. Database design
  4. UI component list
  5. Task file content
```

```
  6. Migration SQL
  7. Implementation steps

Wait for user approval before generating final code.

FEATURE REQUEST:
[paste plain-language description here]
```

8.6b Concrete Example Prompts (PROMPT5 Patterns)

These are the exact prompt patterns from the 3-step loop workflow. Never ask AI to 'build something' directly. Always follow: Plan → Migrate → Implement.

■ The 3-Step Loop

1. Plan feature (task spec) → 2. Generate migration → 3. Implement UI + Supabase queries. This sequence keeps the system stable. Skipping steps causes AI to break things.

PROMPT — Full Feature Prompt — Inventory System Example

```
Read these files before making changes:

/ai/brain.md
/ai/context/product_anchor.md
/ai/context/architecture_anchor.md
/ai/context/database_anchor.md
/ai/context/security_anchor.md

Now implement the feature described below.

FEATURE: Inventory system for characters.

Requirements:
  Users can add items to characters.

  Each item includes:
    - name
    - description
    - quantity
```

```

  Relationships:
    inventory_items → characters

  Security:
    Users may only access items belonging to their characters.

Tasks:
  1. Generate a database migration in /db/migrations/
  2. Implement RLS policies
  3. Update schema.sql
  4. Create API functions in /lib/inventory.ts
  5. Build UI components for adding and listing items
  6. Connect UI to Supabase queries
```

PROMPT — UI-Only Prompt — QuestCard Component

```
Follow:
  /ai/brain.md
  /ai/context/ui_anchor.md

Create a UI component: QuestCard

Requirements:
  Displays:
    - quest name
    - description
    - completion status

  Actions:
    - Mark quest complete (toggle)
    - Delete quest (with confirmation)

  Location: /components/QuestCard.tsx

Then update the quest page (/app/quests/page.tsx) to use it.
```

PROMPT — Add Search to Existing System

```
Follow /ai/brain.md.
```

```

Enhance the character system with search functionality.

Requirements:
  - Search by name (partial match)
  - Filter by class (dropdown)
  - Update Supabase query in /lib/characters.ts
  - Add search input and filter UI to character list page
  - Debounce search requests (300ms delay)

Do not rebuild the entire character system.
Modify only the query function and the list component.
```

PROMPT — One-Shot Feature — Quest System

```
Follow:
  /ai/brain.md
  /ai/security.md
  /ai/context/product_anchor.md
  /ai/context/database_anchor.md

Implement the following feature fully, end to end.

FEATURE: Quest tracking system

Fields:
  - title text required
  - description text optional
  - status text default 'active'
  - world_id uuid FK to worlds
  - user_id uuid FK to auth.users

Actions:
  - create quest
  - update quest
  - delete quest
  - list quests (filtered by world)

Tasks:
```

```
  1. Generate database migration with RLS policies
  2. Update schema.sql
  3. Implement Supabase queries in /lib/quests.ts
  4. Create QuestCard and QuestForm components
  5. Create /app/quests/page.tsx
  6. Connect UI to backend queries
```

PROMPT — Ask Before Building

```
Analyze the repository and propose the best architecture
for implementing this feature.

Do NOT write any code yet.

Output:
  1. Recommended implementation approach
  2. Tables and fields required
  3. Components that would be affected
  4. Potential edge cases
  5. Security considerations
  6. Estimated complexity (Low / Medium / High)

Feature:
[description]

Once I approve the plan, then proceed to implementation.
```

8.6c Remaining Mega Prompt Library (Prompts 19–25)

These are the final entries from the 25-prompt mega library covering automation, data management, access control, and full auditing.

PROMPT — 19 — Background Jobs (Supabase Edge Functions)

```
Follow /ai/brain.md.

Create a scheduled background job using Supabase Edge Functions.

Feature: [describe the scheduled task]

```

```
Requirements:
  - Trigger: [cron schedule, e.g. daily at midnight]
  - Action: [what the job does]
  - Tables affected: [list tables]
  - Store results in: [table name]

Generate:
  - Edge function code
  - Supabase cron schedule configuration
  - Any new migration required
```

PROMPT — 20 — Data Export System

```
Follow /ai/brain.md.

Implement a data export system for: [entity/table]

Requirements:
  - Export format: CSV and JSON
  - Scope: current user's data only (respect RLS)
  - Trigger: button on the [page] page
  - File download: browser download, no server storage

Generate:
  - Export utility function in /lib/export.ts
  - Download button UI component
  - Supabase query that fetches export data
```

PROMPT — 21 — Rate Limiting

```
Follow /ai/brain.md.

Implement rate limiting for the following API actions:
  [list actions, e.g. createCharacter, sendMessage]

Requirements:
  - Use Supabase Edge Functions middleware
  - Limit: [N] requests per [time window] per user
  - Return 429 status when limit exceeded
```

```
  - Store rate limit counters using Upstash Redis (preferred for production — low
latency,
    no DB lock contention; add to integrations spec first using the Integration
Discovery
    Prompt in Section 17.3) or via a Supabase rate_limits table for low-traffic
internal
    APIs where simplicity is more important than throughput

Generate the rate limiting middleware and apply it to the specified actions.
```

PROMPT — 22 — Role-Based Permissions System

```
Follow /ai/brain.md and /ai/security.md.

Implement a role-based permissions system.

Roles:
  admin — full access to all data and admin UI
  moderator — can view and moderate content, cannot delete users
  user — standard access to own data only

Requirements:
  1. Add 'role' column to profiles table (migration)
  2. Update RLS policies to check role where needed
  3. Create a helper function: getUserRole(user_id)
  4. Add role check to admin-only pages
  5. Add admin policy pattern to all relevant tables

Generate migration, updated policies, and role utility function.
```

PROMPT — 23 — Feature Upgrade

```
Follow /ai/brain.md.

Enhance the following feature: [feature name]

Current behavior: [describe current state]

Requested additions:
  - [addition 1]
  - [addition 2]
```

```

Rules:
  - Do not rebuild the feature from scratch
  - Modify existing components and queries only
  - Add a migration only if new columns are needed
  - Preserve all existing functionality
```

PROMPT — 24 — Architecture Review

```
Analyze the current repository architecture.

Identify:
  - Structural improvements (naming, organization, separation of concerns)
  - Missing abstraction layers
  - Components that have grown too large and should be split
  - Queries that should be moved to /lib/ if they're in components
  - Any deviations from the patterns in /ai/brain.md

Do NOT modify any files. Provide recommendations only.
```

PROMPT — 25 — Full Repository Audit

```
Perform a complete repository audit.

Check and report on:
  Security: missing RLS, unsafe queries, open policies
  Performance: missing indexes, N+1 queries, unoptimized components
  Architecture: deviations from brain.md rules
  Completeness: missing error states, loading states, empty states
  Spec drift: code that contradicts /ai/database.md or /ai/ui.md

Output a prioritized report with CRITICAL / HIGH / MEDIUM / LOW severity per issue.
```

8.7 Debugging Prompts

PROMPT — Debug Fix

```
Follow:
  /ai/brain.md
  /ai/debug.md
```

```

Error:
[paste full error message and stack trace]

Fix this issue.

Rules:
  - Do not rewrite unrelated files
  - Explain the root cause before fixing
  - Show only the changed lines, not entire files
  - Verify the fix does not break RLS or auth
```

8.8 Architecture Review & Refactor Prompts

PROMPT — Architecture Review (No Code)

```
Analyze the repository and propose the best architecture
for implementing the following feature.

Do NOT write any code.

Output:
  1. Recommended approach
  2. Tables affected or required
  3. Components affected or required
  4. Potential edge cases
  5. Security considerations

Feature:
[description]
```

PROMPT — Safe Refactor

```
Follow /ai/brain.md and /ai/context/refactor_anchor.md.

Refactor the [system name] system.

Goals:
  - Remove duplicate Supabase queries
```

```
  - Move all database logic to /lib/
  - Simplify components to presentation only
  - Ensure behavior is identical after refactor

Rules:
  - Do not change functionality
  - Do not modify migrations
  - Do not add new dependencies
  - Show before/after diff for each changed file
```

9 Cross-Platform Development

Platform and device scoping, layout rules, overrides, and HTML previews

9.1 Platform & Device Scope

■ Scope Note — AI-Generated Native UI

The cross-platform generation system reliably produces production-quality output for web (React/Next.js). For iOS (SwiftUI), Android (Jetpack Compose), and Desktop (Electron/Tauri), treat AI output as architectural scaffold and typed boilerplate — expect manual refinement for complex components, animations, platform-specific gestures, and native API interactions. The schema layer (Sections 10.1–10.6) is the durable value; the native code output is a vetted starting point, not a finished product.

Not every feature belongs on every platform. Always define scope explicitly before building. AI should never assume identical behavior across devices.

Web	React / Next.js	Primary platform; header nav, keyboard shortcuts, hover states
iOS	SwiftUI	Bottom tab navigation, swipe gestures, push notifications
Android	Jetpack Compose	Material navigation, back button handling, native notifications
Desktop	Tauri (preferred) / Electron	Menu bar, multi-window, larger information density

■ Desktop Framework Note

> Tauri is the recommended choice for new desktop projects as of 2025. It produces significantly smaller bundles (typically 5–10 MB vs 150+ MB for Electron), uses a Rust backend with a stronger security sandbox, and has lower memory overhead. Electron remains viable for teams with existing Node.js backend expertise or who require Node.js-specific APIs.

9.2 AI Cross-Platform Pipeline Prompt

PROMPT — Cross-Platform Feature Prompt

```
You are a Cross-Platform Application Architect.

For every feature you must:
1. Identify which platforms it applies to
2. Identify which device types it applies to
3. Determine if layout differs per device
4. Determine if behavior differs per platform

Platforms: Web / iOS / Android / Windows Desktop / Mac Desktop
Devices: Mobile / Tablet / Laptop / Desktop / Ultra-wide

Before writing code, answer:
  - Which platforms should this feature exist on?
  - Should mobile/tablet/desktop layouts differ?
  - Are any UI elements platform-specific?

Then generate:
  - FEATURE_SPEC.md
  - UI_LAYOUT_RULES.md
  - DEVICE_VARIANTS.md

Feature request:
[description]
```

9.3 Device Layout System & Breakpoints

Mobile	`0 – 600px`	Stacked layout, full-width buttons, bottom navigation
Tablet	`601 – 1024px`	Two-column layout, sidebar optional, larger touch targets
Laptop	`1025 – 1440px`	Standard desktop layout, hover states active
Desktop	`1441 – 1920px`	Full layout, dense information display, multi-panel
Ultra-wide	`1921px+`	Max-width container centered, no full stretch

9.4 Device-Specific Edit Prompt

This prompt modifies one device scope without touching others. Essential for fixing mobile layout bugs without breaking the desktop view.

PROMPT — Device-Scoped Modification

```
You are performing a device-scoped UI modification.

Target Scope:
  Platform: [Web / iOS / Android / Desktop]
  Device: [Mobile / Tablet / Desktop]

You MUST modify only the specified scope.
Do NOT modify other device layouts.

Describe the changes made and show only the affected CSS/component code.

Requested change:
[description]
```

9.5 HTML Preview Generation

PROMPT — Mobile Preview Generator

```
Generate an HTML preview simulating the mobile layout for:
[feature or page name]

Requirements:
```

```
  - Device frame width: 390px
  - Scrollable content
  - Responsive CSS
  - Mobile-appropriate font sizing and spacing

Include:
  - Header / navigation
  - Feature UI
  - Interactive components (non-functional, visual only)

Output: /preview/mobile_preview.html
```

9.6 Multi-Device Preview

PROMPT — Multi-Device Preview Generator

```
Generate an HTML file showing the same UI in multiple device frames.

Frames to include:
  - iPhone (390px)
  - Tablet (768px)
  - Laptop (1280px)
  - Desktop (1440px)

Display all frames side by side with labels.
Use responsive CSS inside each frame.

Output: /preview/responsive_preview.html
```

9.7 Platform Behavior Differences

Define platform-specific behaviors explicitly. Include this in the feature spec or the relevant context anchor.

Platform Behavior Definition

```
# PLATFORM BEHAVIOR SPEC

Feature: Navigation

Web:
```

```
  - Header navigation bar
  - Keyboard shortcuts (Cmd+K for search)
  - Hover states on all interactive elements

iOS:
  - Bottom tab bar (5 items max)
  - Swipe back gesture
  - Push notification permission prompt

Android:
  - Material Design bottom navigation
  - Hardware back button handling
  - Notification channels

Desktop (Electron):
  - Native menu bar integration
  - Multi-window support
  - Larger information density
```

10 Unified UI Schema (UUIS)

Design once, generate for all platforms — the schema-driven UI approach

10.1 Core Concept

■ Platform Generation Note

Generating from the Unified UI Schema produces reliable React/Tailwind output. SwiftUI and Jetpack Compose output should be reviewed by a platform-native developer before shipping. The schema enforces design consistency; platform fidelity requires human verification.

Instead of designing UI separately for each platform, define it once in a platform-agnostic YAML schema. AI generators then produce platform-specific implementations from that single source of truth.

■ Folder Structure

```
ui-schema/
■■■ design_tokens.yaml # Colors, typography, spacing
■■■ components/ # Individual component schemas
■ ■■■ button.yaml
■ ■■■ card.yaml
■■■ features/ # Feature-level UI schemas
■ ■■■ messaging.yaml
■ ■■■ user_profile.yaml
■■■ layouts/ # Layout templates

platform_generators/ # Generated output per platform
■■■ web/ # React + Tailwind output
■■■ ios/ # SwiftUI output
■■■ android/ # Jetpack Compose output
■■■ desktop/ # Electron / Tauri output
```

10.2 Design Tokens

Design tokens are the global style constants that ensure visual consistency across all generated platforms. AI must reference design_tokens.yaml when generating any UI.

■ /ui-schema/design_tokens.yaml

```
# /ui-schema/design_tokens.yaml
```

```yaml
colors:
  primary: "#2563eb"
  secondary: "#10b981"
  background: "#ffffff"
  surface: "#f3f4f6"
  text: "#111827"
  muted: "#6b7280"
  destructive: "#ef4444"

typography:
  font_family: Inter
  scale:
    mobile: 14
    tablet: 16
    desktop: 18
  weights:
    body: 400
    medium: 500
    bold: 700

spacing:
  xs: 4
  sm: 8
  md: 16
  lg: 24
  xl: 40
  2xl: 64

border_radius:
  sm: 4
  md: 8
  lg: 16

breakpoints:
  mobile: 600
  tablet: 1024
  laptop: 1440
  desktop: 1920
```

10.3 Component Schema

- **/ui-schema/components/button.yaml**

```yaml
# /ui-schema/components/button.yaml

component: Button

variants:
  - primary
```

```
  - secondary
  - destructive
  - ghost

props:
  label: string
  icon: optional string
  disabled: boolean default false
  loading: boolean default false
  action: event

layout_rules:
  mobile:
    width: full
    height: 48px
  tablet:
    width: auto
    height: 44px
  desktop:
    width: auto
    height: 40px

platform_notes:
  ios: Use native UIButton haptic feedback
  android: Use Material ripple effect
```

10.4 Feature Schema

- **/ui-schema/features/user_profile.yaml**

```
# /ui-schema/features/user_profile.yaml

feature: User Profile

platforms:
  - web
  - ios
  - android
  - desktop

layout:
  mobile:
    type: stacked
    avatar_size: 80px
  tablet:
    type: two-column
    avatar_size: 100px
  desktop:
    type: sidebar
```

```
    avatar_size: 120px

components:
  - Avatar
  - Username
  - Bio
  - FollowButton
  - MessageButton
  - ActivityFeed

actions:
  - follow_user
  - send_message
  - edit_profile

data_model:
  source_table: profiles
  fields:
    - id, name, bio, avatar_url, created_at
```

10.5 Platform Generation Prompt

PROMPT — Generate All Platform Implementations

```
You are a cross-platform UI generator.

Input: the UI schema at /ui-schema/features/[feature].yaml

Generate implementations for:
  Web → React + Tailwind CSS → /platforms/web/
  iOS → SwiftUI → /platforms/ios/
  Android → Jetpack Compose → /platforms/android/
  Desktop → Electron + React → /platforms/desktop/

Rules:
  - Follow /ui-schema/design_tokens.yaml exactly
  - Apply responsive layouts per device breakpoints
  - Apply platform-specific navigation patterns
  - Output one file per component per platform
```

10.6 Device-Specific Overrides

■ **Device Override Pattern**

```
# Device overrides in feature schema

overrides:
  mobile:
    hero_image:
      alignment: center
      height: 200px
  desktop:
    hero_image:
      alignment: left
      height: 400px

  ios:
    navigation:
      type: bottom_tabs
  android:
    navigation:
      type: material_bottom_nav
  web:
    navigation:
      type: header_nav
```

10.7 Preview, Diff Tooling & Interactive Prototypes

PROMPT — UI Diff Analysis

```
Compare the current generated UI against the UI schema.

Detect:
  - Layout inconsistencies across breakpoints
  - Spacing values not matching design_tokens.yaml
  - Font sizes deviating from token scale
  - Component overflow on mobile
  - Missing responsive behavior

Report each issue with: affected breakpoint, component, and suggested fix.
```

PROMPT — Generate Interactive Prototype

```
Generate an interactive prototype using HTML + JavaScript.

Requirements:
  - Simulate navigation between pages
  - Simulate component interactions (open modal, toggle, form submit)
```

```
  - Use the design tokens from /ui-schema/design_tokens.yaml
  - No real backend calls — mock all data

Output: /preview/interactive_prototype.html

This prototype should be openable in a browser to test UX
before writing real implementation code.
```

11 Full AI App Generator Architecture

The complete pipeline from idea to multi-platform deployed application

11.1 Master Repo Structure

■ Folder Structure

```
repo/
■■■ app-spec/
■ ■■■ product.md
■ ■■■ features.md
■ ■■■ user_roles.md
■■■ ui-schema/
■ ■■■ design_tokens.yaml
■ ■■■ components/
■ ■■■ features/
■■■ backend-schema/
■ ■■■ database.md
■ ■■■ api.md
■ ■■■ services.md
■■■ platforms/
■ ■■■ web/
■ ■■■ ios/
■ ■■■ android/
■ ■■■ desktop/
■■■ preview/
■ ■■■ mobile.html
■ ■■■ tablet.html
■ ■■■ desktop.html
■■■ ai/
    ■■■ prompt_library.md
    ■■■ architecture_rules.md
```

11.2 App Spec Layer

Three files define the product completely before any UI or database work begins.

app-spec/product.md

One-page description: what the app does, who the users are, the core value proposition.

app-spec/features.md

Numbered list of all features grouped by priority: core, secondary, future.

app-spec/user_roles.md

All roles (admin, user, guest, moderator) with their permissions and data ownership rules.

11.3 UI Schema Layer

After product specs are approved, define the UI schema. This layer is platform-agnostic — it describes what exists and how it behaves, not how it is rendered.

11.4 Backend Schema Layer

The backend schema layer defines database tables, API endpoints, and service boundaries before any migration SQL is generated.

■ **backend-schema/services.md**

```
# /backend-schema/services.md
# AI generates microservices or modules from these service definitions.

services:
  auth_service:
    handles: registration, login, logout, password reset
    tables: auth.users, profiles

  messaging_service:
    handles: send, receive, delete messages
    tables: conversations, messages

  notification_service:
    handles: create, deliver, mark-read notifications
    tables: notifications

  media_service:
    handles: upload, retrieve, delete files
    storage: Supabase Storage (user-files bucket)
```

11.5 Platform Generation Layer

After schemas are approved, platform code is generated. Each platform lives in its own subfolder. AI generates one file at a time, per component, per platform.

■ **Generated Platform Files**

```
platforms/
■■■ web/
■ ■■■ components/
■ ■■■ ProfileCard.tsx
```

```
■ ■■■ MessageThread.tsx
■■■ ios/
■ ■■■ Views/
■ ■■■ ProfileView.swift
■ ■■■ MessageView.swift
■■■ android/
■ ■■■ ui/
■ ■■■ ProfileScreen.kt
■ ■■■ MessageScreen.kt
■■■ desktop/
    ■■■ components/
        ■■■ ProfileCard.tsx
        ■■■ MessageThread.tsx
```

11.6 Preview & Testing Layer

Test Stack

■ **Install**

```
# Unit + integration tests
npm install -D vitest @vitest/ui

# Component tests
npm install -D @testing-library/react @testing-library/jest-dom jsdom

# E2E tests (add only if E2E coverage is required — not default)
npm install -D @playwright/test && npx playwright install
```

■ **package.json scripts**

```
{
  "scripts": {
    "test": "vitest run",
    "test:watch": "vitest",
    "test:ui": "vitest --ui",
    "test:e2e": "playwright test"
  }
}
```

/lib/ functions (unit)	`Vitest`	Faster than Jest, native ESM, no config overhead
UI behavior (component)	`@testing-library/react`	Behavior-first, avoids implementation coupling

Supabase integration	`Vitest + test client`	Use a dedicated test project or local Supabase via CLI
Full user flows (E2E)	`Playwright`	Add only when the feature warrants full browser coverage

■ **vitest.config.ts + tests/setup.ts**

```
// /vitest.config.ts
import { defineConfig } from 'vitest/config'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    setupFiles: ['./tests/setup.ts'],
    globals: true,
  },
})

// /tests/setup.ts
import '@testing-library/jest-dom'
```

■ **Integration test pattern**

```
// /tests/integration/characters.test.ts
import { describe, it, expect } from 'vitest'
import { createClient } from '@supabase/supabase-js'
import type { Database } from '@/lib/database.types'

// Use a dedicated test project — never run against production
const supabase = createClient<Database>(
  process.env.TEST_SUPABASE_URL!,
  process.env.TEST_SUPABASE_SERVICE_ROLE_KEY!
)

describe('characters table', () => {
  it('enforces RLS — user cannot read another user's characters', async () => {
    // test implementation
  })
})
```

■ **Never use your production database for integration tests.**

Use a dedicated Supabase test project or a local Supabase instance via the CLI. Store test credentials in .env.test — never in .env or committed to git.

Test Folder Structure

■ Test Structure

```
tests/
■■■ unit/
■ ■■■ [feature].test.ts
■■■ integration/
■ ■■■ [feature].test.ts
■■■ ui/
    ■■■ [component].test.tsx
```

PROMPT — Generate Automated Tests

```
Generate automated tests based on the feature specification at:
  /ai/tasks/[feature].md

Create:
  - Unit tests for all /lib/ query functions
  - Integration tests for Supabase interactions (use mock client)
  - Component tests for UI behavior

Place tests in:
  /tests/unit/[feature].test.ts
  /tests/integration/[feature].test.ts
```

11.7 Full Pipeline Workflow

1	**Idea**	Describe app in plain English
2	**Product Spec**	Run interview prompt → generate /app-spec/ files
3	**Feature Specs**	Define /app-spec/features.md with all features
4	**UI Schema**	Create /ui-schema/ components and feature YAMLs
5	**Backend Schema**	Create /backend-schema/ database, api, services docs
6	**Architecture Plan**	Review all specs with AI; catch edge cases
7	**Migrations**	Generate and apply all migration SQL files

8	**Code Generation**	Generate platform code feature by feature
9	**Preview**	Generate HTML previews for all device sizes
10	**Testing**	Generate and run automated tests
11	**Security Review**	Run security audit prompt
12	**Deploy**	Push to GitHub → auto-deploy via Vercel

12 Site Analyzer Tool

CLI tool for scraping, analyzing, and rebuilding any website

12.1 Tool Goal & Pipeline

The site analyzer takes a URL and produces a structured architecture plan, component map, and clean rebuilt HTML. Useful for cloning UI layouts, migrating systems, or reverse-engineering design patterns from existing sites.

■ Tool I/O

```
# Input
analyze-site https://example.com

# Output
/analysis/
■■■ site_summary.md
■■■ architecture_plan.md
■■■ components.md
■■■ page_structure.json
■■■ design_tokens.json
■■■ rebuilt_html/
    ■■■ index.html
    ■■■ about.html
    ■■■ contact.html
```

Pipeline Steps

1. **Scrape** — Use Puppeteer or Playwright to capture full HTML, CSS, DOM tree, navigation links
2. **Map DOM** — Build simplified component tree: Header → Nav, Hero → Headline + CTA, etc.
3. **AI Analysis** — Feed DOM to AI; get component list, section map, reusable UI, data models
4. **Architecture Plan** — AI generates: pages, components, design system, navigation flow, data needs
5. **Rebuild HTML** — AI creates clean, semantic, tracking-free HTML from the architecture plan

12.2 CLI Design

■ CLI Command

```
# Basic usage
site-analyzer analyze https://example.com

# With options
site-analyzer analyze https://example.com \
  --pages 10 \
  --extract-components \
  --rebuild-html \
  --generate-plan
```

12.3 Node.js Scraper

- **scraper.mjs**

```
#!/usr/bin/env node
import puppeteer from "puppeteer";
import fs from "fs";

const url = process.argv[2];

async function analyzeSite() {
  const browser = await puppeteer.launch();
  const page = await browser.newPage();

  await page.goto(url, { waitUntil: "networkidle2" });

  // Capture full HTML
  const html = await page.content();
  fs.writeFileSync("analysis/site.html", html);

  // Capture all internal links
  const links = await page.$$eval("a", anchors =>
    anchors.map(a => a.href).filter(h => h.startsWith(url))
  );
  fs.writeFileSync("analysis/links.json", JSON.stringify(links, null, 2));

  // Capture screenshot for visual analysis
  await page.screenshot({
    path: "analysis/screenshot.png",
    fullPage: true
  });

  await browser.close();
  console.log("Site captured. Ready for AI analysis.");
}

analyzeSite();
```

12.4 AI Analysis Prompt

PROMPT — Site Analysis

```
You are a senior frontend architect.

Analyze the provided HTML and produce:

1. Website Summary (2-3 sentences)
2. Page List (all pages found)
3. Navigation Structure (how pages connect)
4. UI Component Map (reusable components detected)
5. Design Tokens (primary color, font, spacing scale)
6. Accessibility Issues (critical issues only)
7. Performance Issues (obvious problems)

Then generate:
  SITE_PLAN.md
  COMPONENT_MAP.md
  PAGE_STRUCTURE.json
  DESIGN_TOKENS.json

HTML:
[paste captured HTML or reference analysis/site.html]
```

12.5 HTML Rebuild Prompt

PROMPT — Rebuild Site HTML

```
Using the architecture plan at analysis/architecture_plan.md,
rebuild the website using clean HTML.

Requirements:
  - Semantic HTML5 elements (nav, main, section, article, footer)
  - No tracking scripts, analytics, or third-party tags
  - Responsive layout using CSS Grid and Flexbox
  - WCAG 2.1 AA accessibility compliance
  - All interactive elements keyboard-accessible

Output one HTML file per page in /analysis/rebuilt_html/
Include a shared styles.css referencing the extracted design tokens.
```

★ Advanced Upgrade

After HTML rebuild, prompt AI to convert the site architecture into React components with Tailwind, a Next.js page structure, and a Flutter UI. This turns any existing website into a complete multi-platform specification.

13 AI Project Kickstart

Initialize any new project with full AI-ready architecture in one prompt

13.1 Kickstart Prompt

Run this once at the start of a new project. It generates the entire folder structure and all starter spec files, so you begin coding with AI already anchored to a real architecture.

■ Kickstart vs Interview — Use Both, In Order

The Kickstart Prompt generates folder structure and generic starter spec files. Run this FIRST to create the repository scaffolding. Then run the Product Interview Prompt (Section 1B) to replace the generic spec content with information specific to your actual product. Do not treat the Kickstart output as final spec content — it is scaffolding only.

PROMPT — Project Kickstart

```
You are a senior software architect.

Initialize a new repository optimized for AI-assisted development.

Stack:
  Frontend: Next.js (TypeScript)
  Backend: Supabase
  Database: PostgreSQL
  Hosting: Vercel

STEP 1: Generate the full folder structure including:
  app/, components/, lib/, hooks/, styles/
  db/schema.sql, db/migrations/
  ai/brain.md, ai/product.md, ai/architecture.md
  ai/database.md, ai/security.md, ai/rules.md
  ai/debug.md, ai/checklist.md, ai/lessons.md
  ai/context/ (all four anchors)
  ai/tasks/example_feature.md
```

```
  feature-packs/
  middleware.ts
  lib/supabase.ts
  lib/supabase-server.ts
  lib/env.ts
  lib/database.types.ts (placeholder — populated after first migration)
  lib/schemas/ (folder — one schema file per entity)

STEP 2: Generate starter content for each AI spec file.
  brain.md must include: development workflow,
  migration rules, architecture rules, security requirements.

STEP 3: Create security rules file (security.md) with:
  RLS requirements, user ownership rules, admin permissions.

STEP 4: Generate starter database schema including:
  profiles table with RLS policies.
  Auto-create profile trigger on new user signup.

STEP 5: Generate auth feature pack at feature-packs/auth-system/
  Include: feature.md, schema.sql, policies.sql, tasks.md,
  and stub LoginForm.tsx + RegisterForm.tsx components.

STEP 6: Generate /ai/tasks/example_feature.md showing correct format.

STEP 7: Generate /ai/checklist.md with feature completion checks.

OUTPUT: Return full content of every file, ready to paste.
```

13.2 Post-Kickstart Workflow

1. Review all generated spec files — adjust product.md and architecture.md to match your app
2. Run the Feature Interview prompt (Section 1B) to generate your first set of feature specs
3. Apply the initial schema to Supabase via the SQL editor
4. Install the auth feature pack using the pack installation prompt (Section 7.4)
5. Create your first feature task file in /ai/tasks/
6. Begin feature development using the Master Builder Prompt (Section 8.1)

✓ Edge Case Review Trick

After the kickstart interview, prompt: 'Identify missing edge cases or security risks in this architecture.' AI will often catch privilege escalation vectors, broken RLS configurations, missing indexes, and data leak scenarios that are otherwise easy to miss.

14 Best Practices & Advanced Tricks

Spec locking, AI memory, self-auditing, and iterative development patterns

14.1 Spec Locking

Once your core architecture is stable, add this rule to /ai/brain.md to prevent AI from spontaneously redesigning foundational decisions:

■ **Spec Lock Rule for brain.md**

```
SPEC LOCK

The following files define locked architecture decisions.
Do NOT suggest changes to these without explicit user instruction:

  /ai/architecture.md — stack is fixed
  /ai/database.md — existing tables are final
  /ai/security.md — RLS model is fixed
  /lib/supabase.ts — client is the single instance

If a task requires changing locked architecture,
STOP and ask the user before proceeding.
```

Spec Compliance Rule — Critical Addition

Add this to your /ai/brain.md and to /ai/architecture_rules.md. It forces AI to self-check every implementation against the spec before finishing.

■ **Spec Compliance Rule for brain.md / architecture_rules.md**

```
SPEC COMPLIANCE

Code must always follow the specifications.

If generated code deviates from any spec file, STOP immediately
and report the inconsistency before proceeding.

Specifically:
- If a table is not in /ai/database.md, do not use it
- If a component is not in /ai/ui.md, flag it
- If a query bypasses the Supabase client in /lib/supabase.ts, rewrite it
- If an action violates /ai/security.md, stop and ask

Never silently deviate from the specs. Always surface conflicts.
```

14.2 AI Memory — /ai/lessons.md

Every time AI makes a notable mistake, write it into lessons.md. Reference this file in your brain.md. Over time, this file makes AI significantly more reliable on your specific codebase.

■ **/ai/lessons.md**

```
# AI LESSONS

Lesson 001:
Do not create duplicate Supabase client instances.
Always use the single instance in /lib/supabase.ts.

Lesson 002:
Do not add 'use client' to page.tsx files that fetch data server-side.
Keep data fetching in server components; pass data to client components as props.

Lesson 003:
Always include the user_id column in new tables.
Do not rely on the auth context alone.

Lesson 004:
When creating a form modal, do not import the entire component library.
Use existing modal patterns in /components/ui/Modal.tsx.
```

14.3 Self-Auditing with Checklist

After every feature implementation, run the checklist verification prompt. AI will audit its own output against the requirements.

■ **/ai/checklist.md**

```
# /ai/checklist.md

Before marking any feature complete, verify:

Database:
[ ] Table exists in schema.sql
[ ] RLS is enabled
[ ] All four policies exist (SELECT, INSERT, UPDATE, DELETE)
[ ] Foreign key indexes exist

Backend:
[ ] Query functions exist in /lib/
[ ] TypeScript types defined for all parameters and returns
[ ] Error handling implemented
[ ] Zod schema exists in /lib/schemas/[entity].schema.ts
[ ] All form inputs and API payloads validated before insert/update

UI:
[ ] Component exists in /components/
```

```
[ ] Loading state implemented
[ ] Empty state implemented
[ ] Error state implemented

Integration:
[ ] Page connects to backend queries
[ ] Authentication is enforced (user can only see own data)
[ ] Form validation present

Testing:
[ ] No console.log statements left in code
[ ] No TODO comments left unaddressed
```

14.4 Edge Case & Security Review

PROMPT — Post-Architecture Security Review

```
Review the complete architecture and identify:

1. Privilege escalation risks
   (can a regular user access admin endpoints?)

2. RLS bypass scenarios
   (any query that could return another user's data?)

3. Missing ownership fields
   (tables without user_id or organization_id?)

4. Unsafe API actions
   (delete or update operations without ownership checks?)

5. Data leakage vectors
   (are any fields exposed that should be private?)

6. Storage security
   (are storage buckets properly restricted?)

Report each issue with severity: CRITICAL / HIGH / MEDIUM / LOW
```

■ User-Generated Content — Output Sanitization

Zod validates input shape and types. It does NOT sanitize HTML. If your app renders any user-controlled content as raw HTML — profile bios, rich text editors, markdown-to-HTML output, or comment fields — you must sanitize before rendering.

■ **Install (server-side)**

```
npm install sanitize-html
```

■ **sanitize-html (server-side, recommended)**

```
import sanitizeHtml from 'sanitize-html'

const clean = sanitizeHtml(userContent, {
  allowedTags: ['b', 'i', 'em', 'strong', 'p', 'br'],
  allowedAttributes: {},
})
```

■ **Install (client-side)**

```
npm install dompurify
```

■ **DOMPurify (client-rendered content only)**

```
import DOMPurify from 'dompurify'
const clean = DOMPurify.sanitize(userContent)
```

■ **Add to /ai/rules.md**

All user-generated content rendered as HTML must pass through sanitize-html (server) or DOMPurify (client) before display.

14.5 Iterative Development Rhythm

The most effective cadence for AI-assisted development follows a short sprint loop. Each sprint focuses on one feature, completes it end-to-end, and updates all docs before moving on. Never start the next feature while the current one is partially implemented.

Monday	Define next feature task file in /ai/tasks/
Monday	Generate migration SQL, review, apply to Supabase
Tuesday	Generate backend queries and types in /lib/
Wednesday	Generate UI components and page
Thursday	Run checklist verification prompt, fix gaps

Friday	Run security + performance audit, commit, deploy

15 Platform Migration System

AI-guided migration from any stack to any target platform

15.1 What This System Covers

Platform migration is a structured process for moving an existing application — backend, frontend, or both — to a new technology stack or hosting environment. Migration work is divided into three phases: Audit, Spec, and Execute. No code is moved until both the audit and spec phases are complete and approved.

Backend migrations — Firebase → Supabase, Express → Edge Functions, MySQL → PostgreSQL

Frontend migrations — Vue → Next.js, Create React App → Next.js, Angular → React

Mobile migrations — React Native → Flutter, Cordova → Expo, native → cross-platform

Full-stack migrations — Heroku + MongoDB → Vercel + Supabase

Hosting migrations — AWS → Vercel, DigitalOcean → Railway

15.2 Migration Spec Folder Layout

■ Folder Structure

```
repo/
■■■ migration/
    ■■■ migration_brain.md # AI instruction file for migration work
    ■■■ source_audit.md # Full audit of current system
    ■■■ target_spec.md # Target stack definition
    ■■■ gap_analysis.md # What needs to change and why
    ■■■ risk_register.md # Risks, rollback triggers, mitigation
    ■■■ data_map.md # Source → target data model mapping
    ■■■ api_map.md # Source → target API/route mapping
    ■■■ ui_map.md # Source → target component mapping
    ■■■ cutover_plan.md # Go-live sequence and rollback procedure
    ■■■ tasks/
        ■■■ backend_migration.md
        ■■■ frontend_migration.md
        ■■■ auth_migration.md
        ■■■ storage_migration.md
        ■■■ dns_cutover.md
```

15.3 Migration Brain File

- **/migration/migration_brain.md**

```
# MIGRATION BRAIN

You are a senior migration engineer working on this repository.

Before writing any migration code you MUST read:
  /migration/source_audit.md
  /migration/target_spec.md
  /migration/gap_analysis.md
  /migration/risk_register.md
  /migration/data_map.md
  /ai/security.md

MIGRATION RULES

1. Never delete source code or data until the target is verified live.
2. All schema changes must have a corresponding rollback script.
3. Migrate one system boundary at a time (auth, data, storage, UI).
4. Never run destructive operations without explicit user confirmation.
5. All migrated tables must follow RLS rules in /ai/security.md.
6. Feature parity must be verified against source_audit.md before cutover.
7. External environment variables must be documented in target_spec.md.
8. DNS and traffic cutover is the last step — never first.

MIGRATION PHASES

Phase 1 — Audit: Document source system completely.
Phase 2 — Spec: Define target system and map all components.
Phase 3 — Execute: Migrate in order: data → auth → backend → frontend → storage →
cutover.
Phase 4 — Verify: Run checklist, smoke tests, and security audit.
Phase 5 — Cutover: Switch traffic only after Phase 4 passes.
```

15.4 AI Migration Interview Prompt

Run this prompt at the start of every migration project. It conducts a structured 8-section discovery process and generates all migration spec files on completion.

PROMPT — Platform Migration Interview — Part 1: Source System

```
You are a senior migration architect and infrastructure engineer.

Conduct a complete technical interview about an existing system
and its target platform, then generate all migration spec files.
```

```
Ask ONE section at a time. Summarize answers after each section.
Do NOT generate files until all 8 sections are complete.

SECTION 1: SOURCE SYSTEM — BACKEND
  - Current backend framework and language?
    (Express/Node, Django, Rails, Firebase Functions)
  - Database engine? (MySQL, MongoDB, Firestore, PostgreSQL, DynamoDB)
  - Authentication? (Passport.js, Firebase Auth, Auth0, custom JWT)
  - Background jobs / scheduled tasks? (cron, BullMQ, Celery, Cloud Tasks)
  - Webhooks or event-driven integrations?
  - File storage? (AWS S3, Google Cloud Storage, Cloudinary, local disk)
  - Email sending? (SendGrid, Nodemailer, SES, Postmark)
  - Environment variable names? (names only, not values)
  - Approximate data volume: rows per major table, total storage size.

SECTION 2: SOURCE SYSTEM — FRONTEND
  - Frontend framework? (Vue 2/3, React CRA, Angular, Svelte)
  - SSR, SSG, or pure SPA?
  - CSS framework or design system?
  - State management library?
  - Authentication method? (JWT in localStorage, cookies)
  - API call pattern? (REST, GraphQL, SDK, tRPC)
  - Current build and deployment pipeline?

SECTION 3: SOURCE SYSTEM — MOBILE (if applicable)
  - Platforms? (iOS, Android, both)
  - Framework? (React Native, Flutter, Expo, native Swift/Kotlin)
  - Distribution? (App Store, Play Store, enterprise)
  - Push notifications in use? Which service?
  - Offline support or local data storage?
```

PROMPT — Platform Migration Interview — Part 2: Target & Constraints

```
SECTION 4: TARGET PLATFORM — BACKEND
  - Target backend platform?
    (Supabase, PlanetScale, Railway, Render, AWS Amplify)
  - Target database engine?
  - Auth, background jobs, file storage, email on target?

```

```
SECTION 5: TARGET PLATFORM — FRONTEND
  - Target frontend framework?
  - Target hosting? (Vercel, Netlify, Cloudflare Pages)
  - SSR, SSG, or ISR?
  - Preserve or rebuild the design system?

SECTION 6: TARGET PLATFORM — MOBILE (if applicable)
  - Mobile app required on target stack?
  - Rewrite or adapt existing app?
  - Target framework? Share code with web frontend?

SECTION 7: CONSTRAINTS & RISK
  - Hard cutover deadline?
  - Must source system remain live during migration? (zero-downtime)
  - Rollback window if migration fails?
  - Regulatory requirements? (GDPR, HIPAA, SOC 2)
  - Active users who cannot be disrupted?

SECTION 8: INTEGRATIONS
  - All third-party services currently integrated.
  - Which must be preserved on target?
  - Which can be replaced?

After all 8 sections, generate:
  /migration/source_audit.md /migration/target_spec.md
  /migration/gap_analysis.md /migration/risk_register.md
  /migration/data_map.md /migration/api_map.md
  /migration/ui_map.md /migration/cutover_plan.md
  /migration/tasks/backend_migration.md
  /migration/tasks/frontend_migration.md
  /migration/tasks/auth_migration.md
  /migration/tasks/storage_migration.md
```

15.5 Gap Analysis — /migration/gap_analysis.md

AI generates this file from the interview. It is the decision surface — every incompatibility, required rewrite, and configuration delta is documented here before any code is moved.

Authentication

Firebase Auth (JWT)	Supabase Auth	Migrate users via Admin SDK — Supabase invite or bulk import
Custom session cookies	Supabase Auth	Rewrite auth middleware; map session to auth.uid()

Database

Firestore (document model)	PostgreSQL	Flatten nested documents into relational tables
MySQL INT primary keys	PostgreSQL UUID PKs	Transform PKs during ETL; update all FKs
MongoDB ObjectId	UUID	Generate UUID mapping table; run lookup during migration

Frontend

Vue 3 Composition API	React with hooks	Component-by-component rewrite; map lifecycle hooks
Vuex store	Zustand / React Query	Map store modules to query hooks and local state
Vue Router	Next.js App Router	Map route definitions; implement dynamic segments

15.6 Risk Register — /migration/risk_register.md

CRITICAL	**Risk 001 — Auth Token Invalidation**

Description:	Existing user sessions invalidated when auth provider changes.
Mitigation:	Parallel auth window; email users with re-login instructions.
Rollback Trigger:	Auth failure rate > 5% in first 30 min post-cutover → revert DNS.

HIGH	**Risk 002 — Data Type Mismatch**
Description:	MongoDB BSON types (Decimal128, ObjectId) have no direct PostgreSQL equivalent.
Mitigation:	Write explicit type cast functions in ETL; validate row counts pre/post.
Rollback Trigger:	Row count delta > 0.1% between source and target.

MEDIUM	**Risk 003 — Missing Indexes**
Description:	PostgreSQL query planner will not replicate MongoDB query patterns.
Mitigation:	Run EXPLAIN ANALYZE on 10 highest-traffic queries post-migration.
Rollback Trigger:	P99 API latency increase > 2x baseline within 1 hour of cutover.

HIGH	**Risk 004 — Storage URL Breakage**
Description:	Hardcoded storage URLs in DB records will point to old bucket after migration.
Mitigation:	Run URL rewrite migration script after storage copy is complete.
Rollback Trigger:	Media 404 rate > 1% in first hour post-cutover.

15.7 Backend Migration Task

- `/migration/tasks/backend_migration.md`

```
# TASK: Backend Migration

## PHASE 1 — Schema Migration
1. Generate target PostgreSQL schema from source data model per
/migration/data_map.md.
2. Write migration SQL files in /db/migrations/ following existing naming convention.
```

```
3. Include RLS policies per /ai/security.md on all tables.
4. Add indexes on all foreign keys and high-cardinality query columns.
5. Run migration in Supabase staging environment first.

## PHASE 2 — Data ETL
1. Export source data as JSON or CSV per table.
2. Run transform scripts to handle type conversions and key remapping.
3. Validate row counts and data integrity checksums pre-import.
4. Import into target using Supabase bulk insert or psql COPY.
5. Validate row counts and spot-check 10 random records per table.

## PHASE 3 — Auth Migration
1. Export user records from source auth provider.
2. Use Supabase Admin API: supabase.auth.admin.createUser()
3. Map source user IDs to new Supabase UUIDs in a migration map table.
4. Update all user_id foreign keys using the map table.
5. Drop migration map table after verification.

## PHASE 4 — Business Logic
1. Simple CRUD → replace with Supabase client queries in /lib/.
2. Complex business logic → port to Supabase Edge Functions.
3. Scheduled jobs → configure pg_cron or Supabase scheduled functions.

## PHASE 5 — Storage Migration
1. List all files in source storage bucket.
2. Batch download using source provider SDK.
3. Re-upload to Supabase Storage preserving folder structure.
4. Run URL rewrite script to update stored URLs in database.
5. Verify 100 random file URLs return 200 before cutover.
```

15.8 Migration Execution Prompts

PROMPT — Generate Schema Migration from Source Audit

```
Follow:
  /migration/migration_brain.md
  /migration/source_audit.md
  /migration/data_map.md
  /ai/security.md

Generate PostgreSQL migration files for all tables in /migration/data_map.md.

For each table:
  1. Map source field types to PostgreSQL equivalents.
  2. Generate UUID primary keys (replace INT or ObjectId PKs).
```

```
  3. Enable RLS with all four standard policies.
  4. Add indexes on all FK columns and frequently filtered columns.
  5. Write as /db/migrations/00X_migrate_[table].sql.

Flag any source field types that require manual review.
```

PROMPT — Generate ETL Transform Script

```
Follow /migration/data_map.md.

Generate a Node.js ETL script at /migration/scripts/transform.mjs.

For each table in data_map.md:
  1. Read source JSON export from /migration/exports/[table].json.
  2. Apply field renames and type casts.
  3. Generate new UUID for each record.
  4. Write mapping of old_id → new_uuid to /migration/id_map/[table]_map.json.
  5. Output transformed records to /migration/transformed/[table].json.

Include:
  - Row count validation (input vs output must match).
  - Null checks on required fields.
  - A dry-run mode that validates without writing.

Use only Node.js built-ins and @supabase/supabase-js. No other dependencies.
```

PROMPT — Cutover Verification Checklist

```
Review all migration tasks and verify readiness for cutover.

Check:
  [ ] All tables present in Supabase with correct schema
  [ ] Row counts match source system (within 0.1% tolerance)
  [ ] RLS enabled on all tables; all four CRUD policies present
  [ ] Auth users migrated and verified with spot-check login test
  [ ] Storage files copied and URLs updated in database
  [ ] All env vars set in Vercel / target hosting
  [ ] All third-party integrations re-pointed to new endpoints
  [ ] DNS TTL reduced to 60s at least 24 hours before cutover
  [ ] Rollback DNS record prepared and tested
```

```
  [ ] Monitoring alerts configured on error rate and latency
  [ ] Source system in read-only mode 30 minutes before cutover

Report PASS / FAIL with specific evidence.
Do not approve cutover if any CRITICAL item is FAIL.
```

15.9 Zero-Downtime Cutover Pattern

For live production systems with active users, use this phased traffic-shift sequence rather than a hard cutover. Each phase has a defined rollback trigger.

`1–7 days`	**DUAL-WRITE PHASE**	Source system writes to both source DB and target DB simultaneously. All reads still from source. Validate data parity daily.
`1–2 days`	**READ SHIFT PHASE**	Switch read traffic to target: 10% → 50% → 100% via feature flag. Monitor error rates and latency at each increment. Source remains write-primary.
`15–30 min`	**WRITE CUTOVER**	Enable read-only mode on source. Perform final data sync. Switch all write traffic to target. Verify with smoke tests.
`<5 min`	**DNS CUTOVER**	Update DNS A/CNAME records. TTL must already be 60s. Monitor for 2 hours post-cutover.
`7–30 days`	**SOURCE DECOMMISSION**	Keep source in read-only mode for rollback window. Decommission only after monitoring confirms stability.

16 Advanced Feature Builder

Architecting complex, multi-service features from scratch

16.1 Scope of This Section

The handbook's Task System (Section 6) and Mega Prompt Library (Section 8) cover standard CRUD features effectively. This section addresses advanced, multi-service features that the earlier sections are not designed for.

> **■ Relationship to Existing Content**
>
> Section 8.6 (Feature Interpreter) handles single-entity CRUD features. This section fills the gap for subsystem-scale features requiring service decomposition, external vendor selection, event modeling, error models, and phased implementation. The two complement each other — they do not overlap.

This section applies when a feature involves:

- External service APIs (email delivery, real-time messaging, video, payments)
- Event-driven or queue-based architectures
- Multi-tenant data isolation
- Complex state machines (draft → scheduled → sent → bounced lifecycle)
- Background processing with retry logic
- Features that are effectively standalone subsystems

Examples: transactional email platform, in-app messaging, subscription billing, video upload pipeline, multi-tenant SaaS workspace, AI content generation pipeline.

16.2 Advanced Feature Spec Folder

■ Folder Structure

```
ai/
■■■ features/
    ■■■ [feature_name]/
        ■■■ overview.md # What it does, who uses it, success criteria
        ■■■ service_map.md # Services, vendors, boundaries
        ■■■ state_machine.md # Lifecycle states and transitions
```

```
    ■■■ data_model.md # Tables, fields, relationships
    ■■■ api_design.md # Endpoints, Edge Functions, client queries
    ■■■ event_model.md # Events emitted and consumed
    ■■■ security.md # Feature-specific RLS and access rules
    ■■■ error_model.md # Failure modes and handling strategy
    ■■■ phases.md # Implementation phases in order
    ■■■ tasks/
        ■■■ phase_1.md
        ■■■ phase_2.md
        ■■■ phase_3.md
```

16.3 Advanced Feature Interview Prompt

PROMPT — Advanced Feature Architect Interview — Part 1

```
You are a senior software architect specializing in complex feature systems.

Conduct a structured interview to fully design an advanced feature before
any code is written. Ask ONE section at a time. Summarize after each.
Do NOT generate files until the interview is complete.

Before starting, read:
  /ai/brain.md /ai/product.md /ai/database.md /ai/security.md

SECTION 1: FEATURE OVERVIEW
  - What does this feature do in one sentence?
  - Who uses it? (which roles)
  - Success criteria? (measurable outcome)
  - Does it have a distinct lifecycle? (draft → active → archived)
  - User-facing, internal, or automated?

SECTION 2: EXTERNAL SERVICES
  - External API or service required?
    (email, SMS, payment, video, AI inference, webhooks)
  - Multiple vendor options to evaluate?
  - Vendor webhook or callback mechanism?
  - Node.js/TypeScript SDK available?
  - Rate limits or pricing tiers that affect architecture?

SECTION 3: DATA MODEL
```

```
  - Primary entities introduced?
  - Relationships between entities?
  - Any entity needing status/state field?
  - Soft delete or archiving needed?
  - Shared across tenants or scoped per user?
  - Expected data volume per entity per month?

SECTION 4: EVENT MODEL
  - Events produced that other features should react to?
    (email.sent, payment.failed, message.received)
  - Events consumed from other features?
  - Events stored for audit or replay?
  - Real-time delivery required? (Supabase Realtime)
```

PROMPT — Advanced Feature Architect Interview — Part 2

```
SECTION 5: API SURFACE
  - User-triggered actions? (creates, updates, triggers)
  - Automatic actions? (scheduled, webhook-triggered)
  - Idempotent actions? (safe to retry if called twice)
  - Admin-only actions separate from user actions?
  - Return data for each action?

SECTION 6: ERROR MODEL
  - Expected failure modes?
    (external service down, quota exceeded, invalid input, timeout)
  - Retry strategy for failed operations?
  - Failures surfaced in real-time or asynchronously?
  - Compliance requirements around failure logging?

SECTION 7: SECURITY & PERMISSIONS
  - Available to all users or specific roles?
  - Can users access each other's data in this feature?
  - Resource limits per user? (quotas, rate limits)
  - Handles PII, payment data, or regulated content?

SECTION 8: IMPLEMENTATION PHASING
  - Minimum viable slice of this feature?
  - What can be deferred to a later phase?
```

```
  - Hard dependencies between phases?
  - Target timeline for Phase 1?

After all sections, generate:
  /ai/features/[name]/overview.md
  /ai/features/[name]/service_map.md
  /ai/features/[name]/state_machine.md
  /ai/features/[name]/data_model.md
  /ai/features/[name]/api_design.md
  /ai/features/[name]/event_model.md
  /ai/features/[name]/security.md
  /ai/features/[name]/error_model.md
  /ai/features/[name]/phases.md
  /ai/features/[name]/tasks/phase_1.md
```

16.4 Service Map Example

The service map defines every internal module and external vendor the feature depends on, plus the data flow between them. This is the architectural boundary document.

■ **/ai/features/email_platform/service_map.md**

```
# SERVICE MAP — Email Platform

## Internal Services
| Service | Responsibility | Location |
| email_composer | Draft and template management | /lib/email/composer.ts |
| email_scheduler | Queue and delivery scheduling | Supabase pg_cron + Edge Function
|
| email_tracker | Open/click event ingestion | Supabase Edge Function (webhook) |
| email_quota | Per-user sending limits | /lib/email/quota.ts |

## External Services
| Vendor | Role | Integration Type |
| Resend | Transactional email delivery | REST API + webhook |
| Supabase Storage | Email attachment storage | Supabase JS client |
| Supabase Realtime | Delivery status updates | Supabase channel subscription |

## Data Flow
User → email_composer → emails table (status: draft)
User triggers send → email_scheduler → Resend API
Resend → webhook → email_tracker Edge Function → update emails table
emails table change → Supabase Realtime → UI status update
```

16.5 State Machine — /ai/features/[name]/state_machine.md

Every feature with a lifecycle must have its state machine defined before implementation. Invalid transitions are enforced at the database level, not just in application code.

- **/ai/features/email_platform/state_machine.md**

```
# STATE MACHINE — Email Lifecycle

## States
  draft — Created, not yet queued for delivery
  scheduled — Queued for future delivery
  sending — Handed off to delivery service
  sent — Confirmed delivered to recipient server
  opened — Recipient opened (tracking pixel fired)
  clicked — Recipient clicked a tracked link
  bounced — Delivery failed (hard or soft bounce)
  failed — Internal error prevented delivery attempt
  cancelled — User cancelled before send

## Valid Transitions
  draft → scheduled : user calls scheduleEmail()
  draft → sending : user calls sendNow()
  scheduled → sending : pg_cron job fires at scheduled_at
  scheduled → cancelled : user calls cancelEmail()
  sending → sent : Resend webhook: delivered event
  sending → bounced : Resend webhook: bounce event
  sending → failed : Edge Function unhandled error
  sent → opened : tracking pixel fires
  opened → clicked : tracked link fires

## Invalid Transitions (enforced via DB CHECK constraint)
  sent → draft : BLOCKED
  bounced → sending : BLOCKED
  cancelled → sending : BLOCKED
```

16.6 Phased Implementation Plan

- **/ai/features/email_platform/phases.md**

```
# IMPLEMENTATION PHASES — Email Platform

## Phase 1 — Core Send (MVP)
Scope: compose → send → delivery status
Tables: emails, email_recipients
External: Resend API (send only)
UI: ComposeForm, EmailList, StatusBadge
Deferred: scheduling, tracking, attachments, templates

## Phase 2 — Scheduling & Templates
Scope: schedule for future delivery, reusable templates
```

```
Tables: email_templates
Infra: pg_cron scheduled function
UI: TemplatePicker, SchedulePicker
Depends on: Phase 1 complete

## Phase 3 — Tracking & Analytics
Scope: open/click tracking, bounce handling, delivery dashboard
Tables: email_events
Infra: Resend webhook → Edge Function
UI: EmailAnalytics, EventTimeline
Depends on: Phase 2 complete

## Phase 4 — Attachments & Quotas
Scope: file attachments via Supabase Storage, per-user send limits
Tables: email_attachments
Infra: Storage bucket with RLS
Depends on: Phase 1 (can parallel-track with Phase 2)
```

16.7 Advanced Feature Build Prompts

■ Never Use the Service Role Key in Edge Functions

SUPABASE_SERVICE_ROLE_KEY bypasses Row Level Security entirely — any code running with this key has unrestricted read/write access to your entire database. It is only appropriate for server-side scripts that run in isolation (such as the data migration scripts in Section 18). Edge Functions are callable from the internet. If an Edge Function uses the service role key and contains any vulnerability (injection, logic flaw, missing auth check), your entire database is exposed. Use the anon key or a scoped service account in Edge Functions, and rely on RLS policies to enforce data access. Only use the service role key in offline migration scripts — and revoke it when the migration is complete.

PROMPT — Advanced Feature: Phase Implementation

```
Follow:
  /ai/brain.md
  /ai/security.md
  /ai/context/product_anchor.md
  /ai/context/database_anchor.md
  /ai/features/[name]/overview.md
  /ai/features/[name]/data_model.md
  /ai/features/[name]/service_map.md
  /ai/features/[name]/state_machine.md
```

```
Implement Phase [N] as defined in /ai/features/[name]/phases.md.

STEP 1: Confirm Phase [N] dependencies are met.
STEP 2: Generate all database migrations for Phase [N] tables.
STEP 3: Implement state transition validations as DB CHECK constraints.
STEP 4: Create /lib/[feature]/ module with all query functions.
STEP 5: Create all UI components for Phase [N].
STEP 6: Wire external service calls inside Edge Functions — never client code.
STEP 7: Run /ai/checklist.md verification.

Rules:
  External API keys: Edge Functions or server-side only. Never in client code.
  All external API calls must have error handling and retry logic.
  State transitions must be atomic — use Supabase transactions where
  multiple tables are updated in a single operation.
```

PROMPT — Generate State Machine Migration

```
Follow:
  /ai/security.md
  /ai/features/[name]/state_machine.md
  /ai/features/[name]/data_model.md

Generate a PostgreSQL migration that enforces the state machine at DB level.

Include:
  1. Status column as TEXT with CHECK constraint listing valid states.
  2. A transition validation function that blocks invalid state changes.
  3. A BEFORE UPDATE trigger that calls the validation function.
  4. An audit log INSERT into [feature]_events on every state change.
  5. RLS policies scoped to the owner.

This ensures no application bug or direct DB write can produce
an invalid state transition.
```

PROMPT — Generate Edge Function for External Service

```
Follow:
  /ai/brain.md
  /ai/features/[name]/service_map.md
```

```
  /ai/features/[name]/error_model.md

Generate a Supabase Edge Function at:
  /supabase/functions/[function_name]/index.ts

This function handles:
  [describe the specific integration action]

Requirements:
  - Read API key from Deno.env.get('[ENV_VAR_NAME]') — never hardcode.
  - Validate request payload before calling external API.
  - Exponential backoff retry: 3 attempts, 1s / 2s / 4s delays.
  - On permanent failure: set record status to 'failed', log to [feature]_events.
  - Return: { success: boolean, data?, error? }
  - Never leak vendor error messages directly to the client response.
```

17 Outside Integrations Framework

Discover, select, configure, and implement third-party integrations

17.1 Framework Overview

This system provides a repeatable process for adding any third-party integration. All external credentials are handled exclusively through environment variables. No API key, secret, or webhook signing key is ever committed to the repository.

1. **Discovery** — AI-assisted vendor evaluation and selection
2. **Spec** — Integration spec file defining configuration, events, and contracts
3. **Implementation** — Scaffold code, environment variables, and wiring instructions
4. **User Setup** — Step-by-step instructions for account configuration and credential provisioning

17.2 Integration Spec Folder

■ Folder Structure

```
ai/
■■■ integrations/
    ■■■ integrations_index.md # Master list of all active integrations
    ■■■ [vendor_name]/
        ■■■ spec.md # What, why, and how
        ■■■ env_vars.md # Required environment variables
        ■■■ events.md # Webhooks and events handled
        ■■■ setup_instructions.md # Manual steps in vendor dashboard
        ■■■ test_checklist.md # Verification steps
```

17.3 Integration Discovery Prompt

PROMPT — Integration Discovery & Vendor Selection

```
You are a senior infrastructure architect evaluating third-party integrations.

Before evaluating, read:
  /ai/brain.md /ai/architecture.md /ai/product.md
```

```
I need to integrate: [describe the category]
Examples: transactional email, SMS/OTP, payments, file storage,
          analytics, error monitoring, feature flags, search,
          AI inference, video streaming, push notifications,
          CRM, customer support, scheduling, identity verification

Evaluate the top 3–5 vendors for this category.
For each vendor provide:
  1. Pricing model (free tier, usage-based, seat-based)
  2. Node.js/TypeScript SDK (yes/no + package name)
  3. Webhook/event support (yes/no + event list)
  4. Supabase or Vercel native integration (yes/no)
  5. GDPR / data residency options
  6. Rate limits on free and paid tiers
  7. Reliability SLA (if published)
  8. Best-for summary in one line

Then recommend ONE vendor based on:
  - Stack: Next.js + Supabase + Vercel
  - Expected scale: [user estimate]
  - Budget: [free / low-cost / enterprise]

After vendor selection is confirmed, generate:
  /ai/integrations/[vendor]/spec.md
  /ai/integrations/[vendor]/env_vars.md
  /ai/integrations/[vendor]/events.md
  /ai/integrations/[vendor]/setup_instructions.md
  /ai/integrations/[vendor]/test_checklist.md
```

17.4 Integration Spec Example — Resend

■ **/ai/integrations/resend/spec.md**

```
# INTEGRATION SPEC — Resend (Transactional Email)

## Purpose
Send transactional email (account verification, password reset,
notifications) from the application.

## Package
  npm install resend
```

```
## Integration Point
All email calls must go through a Supabase Edge Function.
Never call the Resend API directly from client-side code.
Edge Function: /supabase/functions/send-email/index.ts

## Data Contract
Input: { to: string, subject: string, template: string, variables: object }
Output: { success: boolean, messageId?: string, error?: string }

## Failure Behavior
  API error: log to email_events, set email status to 'failed'.
  Network timeout: retry up to 3 times with exponential backoff.
  Permanent fail: alert via Supabase DB webhook to admin.

## Webhook Events Consumed
  delivered → emails.status = 'sent'
  bounce → emails.status = 'bounced', log bounce_type
  complaint → flag recipient as unsubscribable
  open → insert into email_events (type: 'open')
  click → insert into email_events (type: 'click', url)
```

17.5 Environment Variables — /ai/integrations/[vendor]/env_vars.md

- **/ai/integrations/resend/env_vars.md**

```
# ENVIRONMENT VARIABLES — Resend

## Required
  RESEND_API_KEY — API key (Settings → API Keys)
  RESEND_WEBHOOK_SECRET — Webhook signing secret (Settings → Webhooks)
  RESEND_FROM_ADDRESS — Verified sender (e.g., noreply@yourdomain.com)
  RESEND_FROM_NAME — Display name (e.g., MyApp)

## Where to Set

### Vercel
  Dashboard → Project → Settings → Environment Variables
  Set separately for: Production, Preview, Development.

### Supabase Edge Functions
  supabase secrets set RESEND_API_KEY=re_xxxxxxxxxxxx
  supabase secrets set RESEND_WEBHOOK_SECRET=whsec_xxxxxxxxxxxx

### Local Development
  Add to .env.local — verify it is in .gitignore
```

```
## Security Rules
  RESEND_API_KEY must NEVER appear in client-side code or browser bundles.
  Use NEXT_PUBLIC_ prefix ONLY for browser-safe variables.
  RESEND_API_KEY must NOT use the NEXT_PUBLIC_ prefix.
```

17.6 Setup Instructions — Manual Vendor Steps

> **■ AI Cannot Perform These Steps**
>
> The following steps require a human to log into the vendor dashboard. AI cannot create accounts, verify domains, or copy API keys. These instructions belong in /ai/integrations/[vendor]/setup_instructions.md so any team member can complete the setup.

■ /ai/integrations/resend/setup_instructions.md

```
# SETUP INSTRUCTIONS — Resend

## Step 1 — Create Account
  Visit https://resend.com, sign up, verify email.

## Step 2 — Add and Verify Sending Domain
  Dashboard → Domains → Add Domain
  Add DNS records provided by Resend (TXT for SPF, CNAME for DKIM).
  Click Verify. Wait up to 72 hours (typically < 30 minutes).

  ■ Do not skip domain verification. Emails from unverified
    domains will be rejected or marked as spam.

## Step 3 — Create API Key
  Dashboard → API Keys → Create API Key
  Name: [project-name]-production | Permission: Sending access
  Copy immediately — shown only once.
  Add to Vercel and Supabase environment variables.

## Step 4 — Configure Webhook
  Dashboard → Webhooks → Add Webhook
  Endpoint: https://[project].supabase.co/functions/v1/resend-webhook
  Select: delivered, bounce, complaint, open, click
  Copy Signing Secret → add as RESEND_WEBHOOK_SECRET.

## Step 5 — Send Test Email
  Dashboard → Emails → Send Test
  Verify delivery. Check SPF/DKIM via mail-tester.com if it lands in spam.
```

17.7 Integration Scaffold Prompt

PROMPT — Generate Integration Scaffold

```
Follow:
  /ai/brain.md
  /ai/integrations/[vendor]/spec.md
  /ai/integrations/[vendor]/env_vars.md
  /ai/integrations/[vendor]/events.md

Generate the complete integration scaffold for [vendor].

Include:
  1. npm install command.
  2. Supabase Edge Function at /supabase/functions/[name]/index.ts
     - Read credentials from environment variables only.
     - Implement data contract from spec.md.
     - Error handling and retry logic.
  3. Webhook handler Edge Function (if events.md defines webhooks)
     - Validate webhook signature before processing.
     - Map each event type to a DB update.
  4. TypeScript types for all inputs and outputs.
  5. Client-side helper at /lib/integrations/[vendor].ts
     - Wraps Edge Function call.
     - Returns typed responses.
     - Handles and normalizes errors.
  6. Any required DB migration for tracking events or state.

Do NOT hardcode any API keys.
```

17.8 Integration Test Checklist

■ **/ai/integrations/resend/test_checklist.md**

```
# TEST CHECKLIST — Resend Integration

## Configuration
  [ ] RESEND_API_KEY set in Vercel (Production + Preview)
  [ ] RESEND_API_KEY set in Supabase secrets
  [ ] RESEND_WEBHOOK_SECRET set in Supabase secrets
  [ ] RESEND_FROM_ADDRESS is a verified sender domain
  [ ] Key not in client bundle: grep -r RESEND_API_KEY .next/

## Send Path
```

```
  [ ] Edge Function deploys: supabase functions deploy send-email
  [ ] Test email sends from staging
  [ ] Email arrives in inbox (not spam)
  [ ] Headers show DKIM and SPF pass (mail-tester.com)
  [ ] emails table record created with correct status

## Webhook Path
  [ ] Webhook URL reachable from public internet
  [ ] Resend dashboard shows webhook as Active
  [ ] Webhook signature validation passes (check Edge Function logs)
  [ ] delivered event → emails.status = 'sent'
  [ ] bounce event → emails.status = 'bounced'

## Error Handling
  [ ] Invalid API key returns structured error, does not crash
  [ ] Retry logic fires on simulated network timeout
  [ ] Failed sends logged to email_events with error details
```

17.9 Integration Index

Resend	Transactional Email	**Active**	`send-email, resend-webhook`
Stripe	Payments	**Active**	`create-checkout, stripe-webhook`
Sentry	Error Monitoring	**Active**	(SDK only — no Edge Function)
PostHog	Product Analytics	Planned	—

✓ Maintain the Index

Every vendor in integrations_index.md must have a spec file, documented env vars, setup instructions, and at minimum a passing send/call test before Status is set to Active. Never mark Active based on configuration alone.

18 User Data Migration System

Importing existing users, content, and media from any source

18.1 What This System Covers

This system handles the structured import of user data, accounts, content, and media from an existing platform or legacy system into the current Supabase-backed application.

■ Legal Requirement

Only use this system for data you have legal authority to migrate: your own data, client data with explicit consent, or organizational data under a data processing agreement. This system does not bypass platform terms of service.

SQL dump	Self-hosted DBs	MySQL .sql, PostgreSQL .sql
CSV / TSV	Most SaaS platforms	Salesforce export, WordPress users CSV
JSON export	Modern SaaS APIs	Intercom, Notion, Airtable
HTML / embedded data	Website scrapes	WordPress HTML export, static site archive
Media archive	Storage providers	S3 bucket, Google Drive folder, ZIP archive
API-based export	Platform APIs	Ghost API, Discourse API, Firebase export

18.2 Migration Workspace Layout

■ Folder Structure

```
repo/
■■■ data-migration/
    ■■■ migration_brain.md # AI instruction file
    ■■■ source_inventory.md # What data exists and in what format
    ■■■ schema_map.md # Source fields → target fields
```

```
    ■■■ transform_rules.md # Normalization, deduplication logic
    ■■■ validation_report.md # Pre-import validation results
    ■■■ import_log.md # Post-import row counts and anomalies
    ■■■ media_manifest.md # File list for media migration
    ■■■ scripts/
    ■ ■■■ extract.mjs # Source data extraction
    ■ ■■■ transform.mjs # Normalization and mapping
    ■ ■■■ validate.mjs # Pre-import validation
    ■ ■■■ import.mjs # Supabase bulk import
    ■ ■■■ media_transfer.mjs # Media file migration
    ■■■ source/ # Raw source data ← GITIGNORED
    ■■■ transformed/ # Ready-to-import data ← GITIGNORED
```

■ Add to .gitignore

Add data-migration/source/ and data-migration/transformed/ to .gitignore immediately. Raw user data — emails, names, IDs — must never be committed to version control.

18.3 Data Migration Interview Prompt

PROMPT — User Data Migration Interview — Part 1: Source & Target

```
You are a senior data migration engineer.

Conduct a structured interview to design a complete, safe data migration
plan. Ask ONE section at a time. Summarize after each section.
Do NOT generate scripts until all sections are complete.

Before starting, read:
  /ai/brain.md /ai/database.md /ai/security.md

SECTION 1: SOURCE DATA INVENTORY
  - Platform or system data is coming from?
  - Data categories? (user accounts, posts, media, comments,
    messages, transactions, audit logs, metadata)
  - Format available? (SQL dump, CSV, JSON, HTML, API, ZIP)
  - Record counts per category? (approx)
  - PII present? (names, emails, phone numbers, addresses)
  - Passwords stored? Hashing algorithm?
    (bcrypt, argon2, MD5, SHA1, plain text)
  - Payment or financial data included?
```

```

SECTION 2: TARGET DATA MODEL
  - Which tables correspond to source entities?
  - All required tables already created with migrations?
  - Source fields → target fields mapping per entity.
  - Fields split, merged, or transformed?
  - Fields with no target equivalent?
    (JSONB metadata column or discard?)

SECTION 3: USER ACCOUNTS & AUTH
  - User accounts being migrated?
  - Source identifier? (email, username, numeric ID)
  - Password reset required or sessions preserved?
  - Social login in source? (Google, GitHub, etc.)
  - Duplicate account plan? (merge, keep newest, keep oldest, flag)
  - Notify users about migration?
```

PROMPT — User Data Migration Interview — Part 2: Media, Quality & Constraints

```
SECTION 4: MEDIA & FILE ASSETS
  - Media types? (images, videos, documents, audio)
  - Total storage volume?
  - Current storage? (S3, local disk, CDN, base64 in DB)
  - Files referenced by URL in database?
    (URLs must be rewritten after migration.)
  - Files larger than 5GB?
    (Requires chunked upload for Supabase Storage.)

SECTION 5: DATA QUALITY
  - Known issues? (nulls, duplicates, encoding problems,
    broken URLs, orphaned foreign keys)
  - Halt on first error or log and continue?
  - Acceptable error tolerance?

SECTION 6: MIGRATION CONSTRAINTS
  - Target must be live during migration?
    (If yes: incremental import, not bulk load.)
  - Maintenance window available?
  - Rollback plan if import fails partway?
```

```
  - Who verifies imported data before go-live?

After all sections, generate:
  /data-migration/source_inventory.md
  /data-migration/schema_map.md
  /data-migration/transform_rules.md
  /data-migration/scripts/extract.mjs
  /data-migration/scripts/transform.mjs
  /data-migration/scripts/validate.mjs
  /data-migration/scripts/import.mjs
  /data-migration/media_manifest.md (if media exists)
  /data-migration/scripts/media_transfer.mjs (if media exists)
```

18.4 Schema Map Example

Entity: Users → profiles

`id`	INT	profiles	external_id	cast to TEXT; keep for lookup
`email`	VARCHAR	profiles	email	lowercase, trim
`first_name`	VARCHAR	profiles	name	concat: first_name + last_name
`created_at`	DATETIME	profiles	created_at	convert to ISO 8601 UTC
`role`	ENUM	profiles	role	map: 'admin'→'admin', 'user'→'user'
`avatar_url`	VARCHAR	profiles	avatar	rewrite domain after storage migration
`plan`	VARCHAR	profiles	metadata	store as JSONB { plan: value }

ID Mapping Table

- **`migration_id_map`**

```
CREATE TABLE migration_id_map (
  entity_type TEXT NOT NULL,
```

```
  source_id TEXT NOT NULL,
  target_uuid UUID NOT NULL,
  PRIMARY KEY (entity_type, source_id)
);

-- Populated during import; used to resolve FK references.
-- Drop after migration is verified complete.
```

18.5 Transform Script

- **/data-migration/scripts/transform.mjs**

```
// /data-migration/scripts/transform.mjs
// Input: /data-migration/source/users.json
// Output: /data-migration/transformed/

import fs from 'fs';
import { randomUUID } from 'crypto';

const source = JSON.parse(fs.readFileSync('./source/users.json', 'utf8'));
const idMap = {}, profiles = [], errors = [];

for (const row of source) {
  try {
    if (!row.email) throw new Error(`Missing email on source id ${row.id}`);
    const uuid = randomUUID();
    idMap[row.id] = uuid;
    profiles.push({
      id: uuid,
      external_id: String(row.id),
      email: row.email.toLowerCase().trim(),
      name: [row.first_name, row.last_name].filter(Boolean).join(' ') || null,
      role: row.role === 'admin' ? 'admin' : 'user',
      created_at: new Date(row.created_at).toISOString(),
      metadata: { plan: row.plan || null,
                  source_platform: 'legacy-system',
                  migrated_at: new Date().toISOString() }
    });
  } catch (err) {
    errors.push({ source_id: row.id, error: err.message });
  }
}

fs.writeFileSync('./transformed/profiles.json', JSON.stringify(profiles, null, 2));
fs.writeFileSync('./transformed/id_map_users.json', JSON.stringify(idMap, null, 2));
fs.writeFileSync('./transformed/errors_users.json', JSON.stringify(errors, null,
2));

console.log(`Transformed: ${profiles.length} profiles, Errors: ${errors.length}`);
```

```
if (errors.length / source.length > 0.005) {
  console.error('ERROR RATE EXCEEDS 0.5% THRESHOLD. Review before importing.');
  process.exit(1);
}
```

18.6 Validation Script

Runs pre-import checks against transformed data. Fails loudly — bad data never reaches the database.

- **/data-migration/scripts/validate.mjs**

```
// /data-migration/scripts/validate.mjs
import fs from 'fs';
const profiles = JSON.parse(fs.readFileSync('./transformed/profiles.json', 'utf8'));
let passed = 0, failed = 0;
const issues = [];

const validators = [
  { name: 'UUID format',
    test: (r) => /^[0-9a-f-]{36}$/.test(r.id) },
  { name: 'Email valid format',
    test: (r) => r.email && /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(r.email) },
  { name: 'Role valid enum',
    test: (r) => ['admin', 'user', 'moderator'].includes(r.role) },
  { name: 'created_at is ISO date',
    test: (r) => !isNaN(Date.parse(r.created_at)) },
  { name: 'No duplicate emails',
    test: (() => {
      const seen = new Set();
      return (r) => { if (seen.has(r.email)) return false; seen.add(r.email); return
true; };
    })() },
];

for (const record of profiles)
  for (const v of validators)
    if (!v.test(record)) { issues.push({ id: record.id, validator: v.name });
failed++; }
    else passed++;

console.log(`Validation: ${passed} passed, ${failed} failed`);
if (failed > 0) {
  fs.writeFileSync('./transformed/validation_issues.json', JSON.stringify(issues,
null, 2));
  console.error('Fix issues before running import.'); process.exit(1);
}
console.log('Validation passed. Safe to run import.');
```

18.7 Import Script

■ SUPABASE_SERVICE_ROLE_KEY — Full Database Access

The import script uses SUPABASE_SERVICE_ROLE_KEY to bypass RLS for bulk inserts. This key has unrestricted access to your entire database. Set it only in your local .env file or a secure CI/CD secrets store. Never commit it to git. Revoke it after the migration is complete if it was provisioned specifically for this operation.

■ `/data-migration/scripts/import.mjs`

```
// /data-migration/scripts/import.mjs
// Prerequisites: validate.mjs must pass with zero failures first.
// Requires: SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY in environment.

import { createClient } from '@supabase/supabase-js';
import fs from 'fs';

// Service role key — bypasses RLS for bulk insert.
// Never expose in client code. Never commit to git.
const supabase = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
);

const profiles = JSON.parse(fs.readFileSync('./transformed/profiles.json', 'utf8'));
const BATCH_SIZE = 500;
let inserted = 0, failed = 0;

for (let i = 0; i < profiles.length; i += BATCH_SIZE) {
  const batch = profiles.slice(i, i + BATCH_SIZE);
  const { error } = await supabase
    .from('profiles')
    .upsert(batch, { onConflict: 'email' }); // upsert = safe to re-run

  if (error) { console.error(`Batch ${i/BATCH_SIZE+1} failed:`, error.message);
failed += batch.length; }
  else { inserted += batch.length; console.log(`${inserted}/${profiles.length}
inserted`); }
}

console.log(`Import complete. Inserted: ${inserted}, Failed: ${failed}`);
fs.writeFileSync('./import_log.json', JSON.stringify({
  run_at: new Date().toISOString(), total: profiles.length, inserted, failed
}, null, 2));
```

18.8 User Account Migration — Auth Handling

Migrating user accounts requires specific handling depending on password portability.

Case A — Email/Password (Source Has bcrypt Hashes)

Supabase does not accept pre-hashed passwords via the Admin API. Create users as passwordless and trigger the password reset flow. After bulk creation, send a migration notification email (via your Section 17 integration) at a controlled rate — not all at once.

```
// Create user via Admin API — no password, triggers reset email
const { data, error } = await supabase.auth.admin.createUser({
  email: profile.email,
  email_confirm: true, // Mark email as verified
  user_metadata: {
    name: profile.name,
    external_id: profile.external_id,
    migrated: true
  }
});
```

Case B — Social Login Migration

If users authenticated via Google, GitHub, etc., create accounts in Supabase Auth and match on email. Supabase will automatically link social logins when the email matches.

Case C — New Accounts Only

If not migrating passwords and treating this as new user onboarding, create accounts with email_confirm: false and trigger the standard Supabase invite email flow.

18.9 Media Migration Script

- **/data-migration/scripts/media_transfer.mjs**

```
// /data-migration/scripts/media_transfer.mjs
// Run after storage bucket and RLS policies are created.
// Uses SUPABASE_SERVICE_ROLE_KEY — see Section 18.7 warning.

import { createClient } from '@supabase/supabase-js';
import fs from 'fs';
import path from 'path';
import mime from 'mime-types';

const supabase = createClient(process.env.SUPABASE_URL,
process.env.SUPABASE_SERVICE_ROLE_KEY);
const BUCKET = 'user-files';
const MEDIA_DIR = './source/media';
const mediaMap = {}; // old_path → new_public_url
```

```
async function transferFile(filePath) {
  const relativePath = path.relative(MEDIA_DIR, filePath);
  const mimeType = mime.lookup(filePath) || 'application/octet-stream';
  const { error } = await supabase.storage.from(BUCKET)
    .upload(relativePath, fs.readFileSync(filePath), { contentType: mimeType,
upsert: true });
  if (error) { console.error(`Failed: ${relativePath}`); return null; }
  const { data } = supabase.storage.from(BUCKET).getPublicUrl(relativePath);
  mediaMap[filePath] = data.publicUrl;
  return data.publicUrl;
}

function getAllFiles(dir) {
  return fs.readdirSync(dir, { withFileTypes: true }).flatMap(e => {
    const p = path.join(dir, e.name);
    return e.isDirectory() ? getAllFiles(p) : [p];
  });
}

const files = getAllFiles(MEDIA_DIR);
console.log(`Found ${files.length} files to transfer.`);
for (const file of files) await transferFile(file);

fs.writeFileSync('./transformed/media_map.json', JSON.stringify(mediaMap, null, 2));
console.log(`Transfer complete. ${Object.keys(mediaMap).length} files uploaded.`);
console.log('Run URL rewrite step next to update database references.');
```

URL Rewrite After Media Transfer

PROMPT — Generate URL Rewrite Migration

```
Follow:
  /ai/brain.md
  /data-migration/schema_map.md
  /data-migration/transformed/media_map.json

Generate a SQL migration that updates all URL fields in the database
to replace source domain URLs with new Supabase Storage URLs.

For each URL column in schema_map.md:
  1. Generate an UPDATE statement using media_map as lookup.
  2. Or if map is too large: generate a Node.js script that reads
     the map and runs batched UPDATE queries via Supabase client.
  3. Verify row count affected matches media_map.json entry count.
```

```
  4. Flag any DB URLs NOT in the media_map (failed transfers).
```

18.10 Post-Migration Verification Prompts

PROMPT — Migration Validation Report

```
Review migration outputs and generate a validation report.

Check:
  [ ] profiles row count matches source (±0.5% tolerance)
  [ ] No duplicate emails:
      SELECT email, COUNT(*) FROM profiles
      GROUP BY email HAVING COUNT(*) > 1
  [ ] All user_id FKs resolve:
      SELECT COUNT(*) FROM posts
      WHERE user_id NOT IN (SELECT id FROM profiles)
  [ ] No nulls in required fields (email, created_at, role)
  [ ] Spot-check: 10 migrated users can authenticate
  [ ] Spot-check: 20 media URLs return HTTP 200
  [ ] No source domain URLs in DB:
      SELECT COUNT(*) FROM posts WHERE content LIKE '%olddomain.com%'
  [ ] migration_id_map table can be safely dropped

Output: /data-migration/validation_report.md
Report each check as PASS / FAIL with row counts as evidence.
```

PROMPT — Generate Rollback Script

```
Follow /data-migration/schema_map.md.

Generate a rollback SQL script at /data-migration/scripts/rollback.sql.

The rollback script must:
  1. TRUNCATE all tables populated during migration in reverse FK order.
  2. NOT drop tables or modify schema — data removal only.
  3. Be idempotent (safe to run multiple times).
  4. Log rollback timestamp to a migration_log table.

This script is run only if migration must be aborted after partial import.
```

```
It returns the database to pre-migration state for a clean restart.
```

18.11 HTML and Raw Text Source Handling

When the source is HTML (WordPress export, static site archive) or raw structured text, use a two-step process: AI extraction first, then the standard transform and import.

PROMPT — Extract Structured Data from HTML Source

```
You are a data extraction specialist.

Input: HTML file or directory at /data-migration/source/html/

Extract all structured data and produce JSON output.

For each page or record found, extract:
  - All identifiable entity fields (names, emails, dates, content, IDs)
  - Page/record type (user profile, post, product, event)
  - Embedded media references (img src, video src, download links)
  - Internal links between records

Output:
  /data-migration/source/extracted_[entity].json per entity type.
  /data-migration/source/media_references.json for all media URLs.

Rules:
  - Ambiguous fields: include with note in a _raw field.
  - Do not discard any data — transformation is the next step.
  - Flag records where required fields (email, name) are missing.
  - Report extraction confidence: HIGH / MEDIUM / LOW per entity type.

After extraction, run the standard transform prompt against the JSON.
```

19 Quick-Start Implementation Reference

Complete sequence — initialize, spec, build, secure, and ship

■ How to Use This Section

This is the executable version of the entire guide. No explanatory prose — only code blocks, file paths, and action steps in order. Work through each Phase sequentially on a new project. Phases 3.2 and 3.3 depend on the SSR client and middleware setup covered in detail in Section 2.6b and 2.6c.

PHASE 1 — Initialize Repository

1.0 Prerequisites & Project Initialization

■ Terminal

```
# 1. Create the Next.js app (TypeScript + App Router)
npx create-next-app@latest my-app --typescript --app --no-src-dir
cd my-app

# 2. Install Supabase packages
npm install @supabase/supabase-js @supabase/ssr

# 3. Install Zod for input validation and env checking
npm install zod

# 4. Install testing dependencies
npm install -D vitest @vitest/ui @vitejs/plugin-react
npm install -D @testing-library/react @testing-library/jest-dom jsdom

# 5. Install Supabase CLI (if not already installed)
npm install -g supabase

# 6. Create your Supabase project at https://supabase.com
# Then link the CLI to your project:
supabase login
supabase init
supabase link --project-ref YOUR_PROJECT_ID
# (YOUR_PROJECT_ID is found in Supabase Dashboard → Project Settings → General)

# 7. Create your environment file
# Create .env.local at the project root with:
```

```
# NEXT_PUBLIC_SUPABASE_URL=https://YOUR_PROJECT_ID.supabase.co
# NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
# (Both values are in Supabase Dashboard → Project Settings → API)
```

■ Ensure .env.local is in your .gitignore

The default Next.js .gitignore includes it, but verify before your first commit. Never commit API keys or Supabase credentials to version control.

■ Verify your .gitignore includes these entries

Add to the default Next.js .gitignore:

■ .gitignore additions

```
# Environment files — never commit
.env.local
.env.test
.env*.local

# Supabase local dev
.supabase/

# Data migration — raw user data must never be committed
data-migration/source/
data-migration/transformed/
```

1.1 Create Folder Structure

■ Folder Structure

```
repo/
■■■ app/ # Next.js pages
■■■ components/ # UI components
■■■ lib/ # Utilities, Supabase clients, query functions
■ ■■■ env.ts # Environment variable validation
■ ■■■ supabase.ts # Browser client
■ ■■■ supabase-server.ts # SSR client (server components + actions)
■ ■■■ database.types.ts # Generated types (stub until first migration)
■ ■■■ schemas/ # Zod validation schemas
■■■ hooks/ # React custom hooks
■■■ styles/ # Global CSS
■■■ middleware.ts # Session refresh — required at root
■■■ db/
■ ■■■ schema.sql # Current full schema
■ ■■■ migrations/ # Numbered migration files
■■■ ai/
■ ■■■ brain.md # Master AI instruction file
■ ■■■ product.md # What the app does
```

```
■ ■■■ architecture.md # Tech stack and data flow
■ ■■■ database.md # All tables and relationships
■ ■■■ security.md # RLS rules and auth model
■ ■■■ checklist.md # Feature completion checklist
■ ■■■ migration_checklist.md
■ ■■■ context/ # Short anchor summaries
■ ■ ■■■ product_anchor.md
■ ■ ■■■ architecture_anchor.md
■ ■ ■■■ database_anchor.md
■ ■ ■■■ security_anchor.md
■ ■■■ tasks/ # One file per feature
■■■ feature-packs/
```

1.2 Run Kickstart Prompt in Claude

PROMPT — Project Kickstart

```
You are a senior software architect.

Initialize a new repository optimized for AI-assisted development.

Stack:
  Frontend: Next.js (TypeScript)
  Backend: Supabase
  Database: PostgreSQL
  Hosting: Vercel

STEP 1: Generate the full folder structure including:
  app/, components/, lib/, hooks/, styles/
  db/schema.sql, db/migrations/
  ai/brain.md, ai/product.md, ai/architecture.md
  ai/database.md, ai/security.md, ai/rules.md
  ai/debug.md, ai/checklist.md, ai/lessons.md
  ai/context/ (all four anchors)
  ai/tasks/example_feature.md
  feature-packs/
  middleware.ts
  lib/supabase.ts
  lib/supabase-server.ts
  lib/env.ts
  lib/database.types.ts (placeholder — populated after first migration)
```

```
  lib/schemas/ (folder — one schema file per entity)

STEP 2: Generate starter content for each AI spec file.
  brain.md must include: development workflow,
  migration rules, architecture rules, security requirements.

STEP 3: Create security rules file (security.md) with:
  RLS requirements, user ownership rules, admin permissions.

STEP 4: Generate starter database schema including:
  profiles table with RLS policies.
  Auto-create profile trigger on new user signup.

STEP 5: Generate auth feature pack at feature-packs/auth-system/.

STEP 6: Generate /ai/tasks/example_feature.md showing correct format.

STEP 7: Generate /ai/checklist.md with feature completion checks.

OUTPUT: Return full content of every file, ready to paste.
```

PHASE 2 — Define Spec Files

2.1 /ai/brain.md

This is the canonical brain.md. The version in Section 2.2 is for reference — this Quick-Start version is what goes in your project.

- **/ai/brain.md**

```
# AI PROJECT BRAIN

You are a senior software engineer working on this repository.

Before writing code you MUST read:
  /ai/product.md
  /ai/architecture.md
  /ai/database.md
  /ai/ui.md
  /ai/api.md
  /ai/rules.md
  /ai/security.md
```

```
Do not skip this step.

--------------------------------------------------
DEVELOPMENT RULES

1. Never modify the database schema without updating /db/schema.sql
2. All database access must use the Supabase client in /lib/supabase.ts (browser)
    or /lib/supabase-server.ts (server) — never create additional client instances
3. UI components belong in /components
4. Pages belong in /app
5. Prefer editing existing components instead of creating duplicates
6. Never introduce new frameworks or libraries unless explicitly requested
7. Do not modify unrelated files
8. Follow existing code style
9. Respect Supabase Row Level Security
10. Keep implementations simple
11. Server components and server actions must use /lib/supabase-server.ts, never
/lib/supabase.ts
12. middleware.ts handles session refresh — never remove, bypass, or exclude from the
matcher
13. Always use the Database generic type from /lib/database.types.ts with all
Supabase clients
14. Regenerate /lib/database.types.ts after every migration before writing new
queries
15. All user inputs must be validated with Zod before any database operation
16. Validation schema files live in /lib/schemas/[entity].schema.ts
```

2.2 /ai/product.md (Template — replace with your product)

- **/ai/product.md**

```
# PRODUCT SPEC

This application is a campaign manager for tabletop RPGs.

Users can:
- Create and manage game worlds
- Add towns and locations to worlds
- Create and track characters
- Log party events and encounters

Primary Entities:
  World
  Town
  Character
  Event

User Types:
  admin — full access
  user — owns their own worlds
```

2.3 /ai/architecture.md (Template)

- **/ai/architecture.md**

```
# ARCHITECTURE

Frontend: Next.js (React, TypeScript)
Backend: Supabase (PostgreSQL, Auth, Storage, Edge Functions)
Hosting: Vercel
Database: PostgreSQL via Supabase

Data Flow:
  Client → Supabase JS Client → PostgreSQL

Auth:
  Supabase Auth (email/password + magic link)

Environment Variables:
  NEXT_PUBLIC_SUPABASE_URL
  NEXT_PUBLIC_SUPABASE_ANON_KEY
```

2.4 /ai/security.md

- **/ai/security.md**

```
# SUPABASE SECURITY MODEL

Authentication is handled using Supabase Auth.
Users are identified by: auth.uid()

All user-owned data must include:
  user_id uuid references auth.users(id)

--------------------------------------------------
GENERAL SECURITY RULES

1. Row Level Security must be enabled on ALL tables.
2. Users may only access rows where: user_id = auth.uid()
3. Admin users may access all rows.
4. Never allow anonymous public read access unless explicitly required.
5. Insert operations must verify ownership.
6. Updates must only modify rows owned by the user.

--------------------------------------------------
CRITICAL RULE

Never create policies that allow:
  TRUE
or
  auth.role() = 'anon'
unless explicitly instructed.
```

2.5a /ai/context/product_anchor.md

- **/ai/context/product_anchor.md**

```
# PRODUCT ANCHOR

App: [Your App Name]

Users can:
- [core action 1]
- [core action 2]
- [core action 3]

Primary entities: [table1], [table2], [table3]
```

2.5b /ai/context/architecture_anchor.md

- **/ai/context/architecture_anchor.md**

```
# ARCHITECTURE ANCHOR

Stack: Next.js + Supabase + Vercel

Auth: Supabase Auth (auth.uid() identifies users)

Client communicates with DB using:
  /lib/supabase.ts — browser client (use client components)
  /lib/supabase-server.ts — server client (server components, actions, routes)
```

2.5c /ai/context/database_anchor.md

- **/ai/context/database_anchor.md**

```
# DATABASE ANCHOR

Core Tables:
  [table1], [table2], [table3]

Relationships:
  [table1] → [table2] ([foreign_key])

All user-owned tables include:
  user_id uuid references auth.users(id)
```

2.5d /ai/context/security_anchor.md

- **/ai/context/security_anchor.md**

```
# SECURITY ANCHOR

RLS enabled on ALL tables.

Users access only rows where:
  user_id = auth.uid()
```

```
Admins access all rows.
Never use: USING (TRUE) or auth.role() = 'anon'
```

PHASE 3 — Database Setup

3.0 /lib/env.ts — Environment Variable Validation

Install: already included with Zod. Add this before your Supabase clients to catch missing environment variables at startup rather than at runtime.

- **/lib/env.ts**

```
// /lib/env.ts
import { z } from 'zod'

const envSchema = z.object({
  NEXT_PUBLIC_SUPABASE_URL: z.string().url('NEXT_PUBLIC_SUPABASE_URL must be a valid URL'),
  NEXT_PUBLIC_SUPABASE_ANON_KEY: z.string().min(1, 'NEXT_PUBLIC_SUPABASE_ANON_KEY is required'),
})

const parsed = envSchema.safeParse({
  NEXT_PUBLIC_SUPABASE_URL: process.env.NEXT_PUBLIC_SUPABASE_URL,
  NEXT_PUBLIC_SUPABASE_ANON_KEY: process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
})

if (!parsed.success) {
  console.error('■ Missing or invalid environment variables:')
  console.error(parsed.error.flatten().fieldErrors)
  throw new Error('Environment variable validation failed. Check .env.local.')
}

export const env = parsed.data
```

3.1 /lib/supabase.ts — Browser Client

- **/lib/supabase.ts**

```
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from './database.types'
import { env } from './env'

export const supabase = createBrowserClient<Database>(
  env.NEXT_PUBLIC_SUPABASE_URL,
  env.NEXT_PUBLIC_SUPABASE_ANON_KEY
)
```

3.2 /lib/supabase-server.ts — Server/SSR Client

- **/lib/supabase-server.ts**

```
// /lib/supabase-server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
import type { Database } from './database.types'
import { env } from './env'

export async function createSupabaseServerClient() {
  const cookieStore = await cookies()
  return createServerClient<Database>(
    env.NEXT_PUBLIC_SUPABASE_URL,
    env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      cookies: {
        getAll() { return cookieStore.getAll() },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // Server Components: middleware handles token refresh
          }
        },
      },
    }
  )
}
```

3.3 /middleware.ts — Session Refresh

- **/middleware.ts**

```
// /middleware.ts
import { createServerClient } from '@supabase/ssr'
import { NextResponse, type NextRequest } from 'next/server'
import { env } from './lib/env'

export async function middleware(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request })

  const supabase = createServerClient(
    env.NEXT_PUBLIC_SUPABASE_URL,
    env.NEXT_PUBLIC_SUPABASE_ANON_KEY,
    {
      cookies: {
        getAll() { return request.cookies.getAll() },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) => request.cookies.set(name, value))
```

```
          supabaseResponse = NextResponse.next({ request })
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          )
        },
      },
    }
  )

  await supabase.auth.getUser() // DO NOT REMOVE — triggers session refresh
  return supabaseResponse
}

export const config = {
  matcher: [
    '/((?!_next/static|_next/image|favicon.ico|.*\.(?:svg|png|jpg|jpeg|gif|webp)$).*
)',
  ],
}
```

3.4 /lib/database.types.ts — Stub

■ **/lib/database.types.ts (stub)**

```
// Stub — replace after first migration with:
// supabase gen types typescript --project-id YOUR_PROJECT_ID > lib/database.types.ts
export type Database = {
  public: {
    Tables: Record<string, never>
    Views: Record<string, never>
    Functions: Record<string, never>
  }
}
```

3.5 Initial Schema — Run in Supabase SQL Editor

■ **Initial Schema — Supabase SQL Editor**

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text,
  name text,
  role text default 'user' check (role in ('admin', 'user', 'moderator')),
  avatar text,
  created_at timestamptz default now()
);

alter table profiles enable row level security;

create policy "Users read own profile"
  on profiles for select using (id = auth.uid());
```

```
create policy "Users update own profile"
  on profiles for update using (id = auth.uid());

-- Auto-create profile on signup
create or replace function public.handle_new_user()
returns trigger as $$
begin
  insert into public.profiles (id, email) values (new.id, new.email)
  on conflict (id) do nothing;
  return new;
exception when others then
  -- Log the error but allow user creation to proceed.
  -- A missing profile is recoverable; a failed signup is not.
  raise warning 'handle_new_user: profile insert failed for user %: %', new.id,
sqlerrm;
  return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();

-- Helper function: used in all admin RLS policies.
-- SECURITY DEFINER prevents infinite recursion when applied to the profiles table.
create or replace function public.is_admin()
returns boolean as $$
  select role = 'admin'
  from public.profiles
  where id = auth.uid();
$$ language sql security definer stable;
```

3.6 Migration Checklist — Verify Before Every Migration

- **/ai/migration_checklist.md**

```
Before finalizing any migration, verify:

[ ] Table has a UUID primary key with default gen_random_uuid()
[ ] RLS enabled on the table
[ ] SELECT policy exists
[ ] INSERT policy with WITH CHECK exists
[ ] UPDATE policy exists
[ ] DELETE policy exists
[ ] Foreign key columns have indexes
[ ] No destructive DROP or TRUNCATE operations
[ ] schema.sql updated to reflect current state
[ ] Run: supabase gen types typescript --project-id YOUR_PROJECT_ID >
lib/database.types.ts
```

```
[ ] Commit updated database.types.ts alongside the migration file
```

PHASE 4 — First Feature

4.1 Task File Template — /ai/tasks/[feature].md

- **/ai/tasks/[feature].md template**

```
# TASK: [Feature Name]

## GOAL
[One sentence describing what this feature does for the user.]

## DATABASE
Tables involved:
- [table_name] — [brief description]

Fields required (if new):
- field_name type [nullable/required]

## UI
Components required:
- [ComponentName] — [what it does]

Pages affected:
- /app/[page]/page.tsx

## VALIDATION (Zod)
- Schema file: /lib/schemas/[entity].schema.ts
- Fields to validate: [field list]

## BACKEND
Query functions required in /lib/[feature].ts:
- [functionName(params)] → return type

## BEHAVIOR
Step-by-step user workflow:
1. User does X
2. System does Y
3. Result Z is shown

## API ACTIONS
Functions to implement:
- actionName(params) → return type

## DONE WHEN
- [ ] User can [primary action]
- [ ] Data persists to database
- [ ] Only the user's own data is visible
```

```
- [ ] Error states handled

## NOTES
Any edge cases, permissions, or special constraints.
```

4.2 Run Master Builder Prompt

PROMPT — Master Builder — Build the Feature

```
You are a senior full-stack engineer responsible for implementing
features in this repository.

Before writing any code you MUST read:
  /ai/brain.md
  /ai/security.md
  /ai/context/product_anchor.md
  /ai/context/architecture_anchor.md
  /ai/context/database_anchor.md

Follow all rules defined in those files.

DEVELOPMENT PROCESS

STEP 1 — Understand the feature
  Explain the feature and what system parts are affected.

STEP 2 — Database planning
  If a DB change is required, create a new migration in /db/migrations/.

STEP 2b — Validation schema
  Create Zod schema in /lib/schemas/[entity].schema.ts.
  Validate all user inputs before any database operation.

STEP 3 — Security planning
  Ensure all tables follow RLS rules in /ai/security.md.

STEP 4 — Backend implementation
  Create or update queries in /lib.

STEP 5 — UI implementation
```

```
  Create or modify components in /components.

STEP 6 — Integration
  Wire components to queries. Update pages in /app.

STEP 7 — Verification
  Run /ai/checklist.md. Report every item as PASS or FAIL.

Feature to implement:
[paste task file content or description]
```

4.3 Run After Every Feature — Checklist Verification

- **/ai/checklist.md**

```
# /ai/checklist.md

Before marking any feature complete, verify:

Database:
[ ] Table exists in schema.sql
[ ] RLS is enabled
[ ] All four policies exist (SELECT, INSERT, UPDATE, DELETE)
[ ] Foreign key indexes exist

Backend:
[ ] Query functions exist in /lib/
[ ] TypeScript types defined for all parameters and returns
[ ] Error handling implemented
[ ] Zod schema exists in /lib/schemas/[entity].schema.ts
[ ] All form inputs and API payloads validated before insert/update

UI:
[ ] Component exists in /components/
[ ] Loading state implemented
[ ] Empty state implemented
[ ] Error state implemented

Integration:
[ ] Page connects to backend queries
[ ] Authentication is enforced (user can only see own data)
[ ] Form validation present

Testing:
[ ] No console.log statements left in code
[ ] No TODO comments left unaddressed
```

PHASE 5 — Security & Launch

5.1 Run Security Audit Before Every Deploy

PROMPT — Security Audit

```
Review the repository using /ai/security.md.

Identify and report:
  - Tables missing RLS enablement
  - Tables missing any of the four standard policies
  - Queries that bypass RLS
  - Potential data leaks between users
  - Storage buckets without policies
  - Any use of USING (TRUE) or anon access

Output a prioritized list of issues with suggested fixes.

For a full post-architecture security review, use the
Post-Architecture Security Review prompt in Section 14.4.
```

■ CSRF Protection — Route Handlers vs Server Actions

Server Actions ('use server' functions) have CSRF protection built into Next.js — they verify the request Origin header automatically. Custom Route Handlers (/app/api/*/route.ts) do NOT have automatic CSRF protection. Any mutating route handler (POST, PATCH, DELETE) should verify the Origin header manually:

■ Route Handler CSRF check

```
// /app/api/example/route.ts
import { NextRequest, NextResponse } from 'next/server'

export async function POST(request: NextRequest) {
  const origin = request.headers.get('origin')
  const allowedOrigin = process.env.NEXT_PUBLIC_SITE_URL

  if (origin !== allowedOrigin) {
    return NextResponse.json({ error: 'Forbidden' }, { status: 403 })
  }
  // ... rest of handler
}
```

Best practice: prefer Server Actions over Route Handlers for all data mutations. Reserve Route Handlers for public webhooks (which use signature verification instead — see Section 17.7).

5.2 Run Performance Audit

PROMPT — Performance Audit

```
Follow /ai/brain.md.

Analyze the repository for performance issues.

Identify:
  - Queries missing indexes on foreign keys
  - N+1 query patterns in components
  - Unnecessary React re-renders
  - Missing pagination on list queries
  - Unoptimized Supabase queries

Suggest specific fixes for each issue found.
```

5.3 Add Security Headers to next.config.ts

Add HTTP security headers to your Next.js config before deploying. These protect against clickjacking, MIME-type sniffing, and restrict which resources the browser will load.

■ **next.config.ts**

```
// next.config.ts
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
  async headers() {
    return [
      {
        source: '/(.*)',
        headers: [
          { key: 'X-Frame-Options', value: 'DENY' },
          { key: 'X-Content-Type-Options', value: 'nosniff' },
          { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
          { key: 'Permissions-Policy', value: 'camera=(), microphone=(),
geolocation=()' },
          // Adjust CSP to match your own external sources (fonts, analytics, etc.)
          {
            key: 'Content-Security-Policy',
            value: [
              "default-src 'self'",
              "script-src 'self' 'unsafe-inline'",
              "style-src 'self' 'unsafe-inline'",
              "connect-src 'self' " + process.env.NEXT_PUBLIC_SUPABASE_URL,
              "img-src 'self' data: blob:",
```

```
              "font-src 'self'",
              "frame-ancestors 'none'",
            ].join('; '),
          },
        ],
      },
    ]
  },
}

export default nextConfig
```

> ■ **Add this to /ai/rules.md**
>
> Never remove or relax security headers in next.config.ts without explicit instruction.

5.4 Full Repository Audit — Run at Each Project Milestone

PROMPT — 25 — Full Repository Audit

```
Perform a complete repository audit.

Check and report on:
  Security: missing RLS, unsafe queries, open policies
  Performance: missing indexes, N+1 queries, unoptimized components
  Architecture: deviations from brain.md rules
  Completeness: missing error states, loading states, empty states
  Spec drift: code that contradicts /ai/database.md or /ai/ui.md

Output a prioritized report with CRITICAL / HIGH / MEDIUM / LOW severity per issue.
```

5.5 Weekly Development Cadence

Monday	Define next feature task file in /ai/tasks/
Monday	Generate migration SQL, review, apply to Supabase
Tuesday	Generate backend queries and types in /lib/

Wednesday	Generate UI components and page
Thursday	Run checklist verification prompt, fix gaps
Friday	Run security + performance audit, commit, deploy

5.6 GitHub Actions — Minimal CI Pipeline

Create .github/workflows/ci.yml to run tests on every pull request:

■ `.github/workflows/ci.yml`

```yaml
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'npm'

      - run: npm ci

      - name: Run unit tests
        run: npm test
        env:
          NEXT_PUBLIC_SUPABASE_URL: ${{ secrets.TEST_SUPABASE_URL }}
          NEXT_PUBLIC_SUPABASE_ANON_KEY: ${{ secrets.TEST_SUPABASE_ANON_KEY }}
```

■ GitHub Secrets

Add TEST_SUPABASE_URL and TEST_SUPABASE_ANON_KEY to your GitHub repository secrets (Settings → Secrets and Variables → Actions). These should point to a dedicated test Supabase project — never your production project.

■ Vercel Preview Environments

> Vercel creates a preview deployment for every PR. Each preview should connect to your staging Supabase project, not production. Set environment variables separately per environment in Vercel Dashboard → Project → Settings → Environment Variables, scoping NEXT_PUBLIC_SUPABASE_URL and NEXT_PUBLIC_SUPABASE_ANON_KEY to 'Preview' separately from 'Production.'

PHASE 6 — Add Feature Packs (Optional)

6.1 Install a Feature Pack

PROMPT — Install Feature Pack

```
Follow /ai/brain.md.

Integrate the feature pack at:
  /feature-packs/[pack-name]/

Tasks:
1. Review schema.sql — do not apply if table already exists
2. Generate migration from schema.sql as:
   /db/migrations/00X_[pack_name].sql
3. Integrate UI components from components/ into /components
4. Connect Supabase queries using /lib/supabase.ts
5. Update /ai/database.md with new tables
```

6.2 Auth System Pack SQL

- **`feature-packs/auth-system/schema.sql`**

```
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  email text,
  name text,
  role text default 'user' check (role in ('admin', 'user', 'moderator')),
  avatar text,
  created_at timestamptz default now()
);

alter table profiles enable row level security;

create policy "Users read own profile"
  on profiles for select using (id = auth.uid());
```

```
create policy "Users update own profile"
  on profiles for update using (id = auth.uid());

create or replace function public.handle_new_user()
returns trigger as $$
begin
  insert into public.profiles (id, email) values (new.id, new.email)
  on conflict (id) do nothing;
  return new;
exception when others then
  -- Log the error but allow user creation to proceed.
  -- A missing profile is recoverable; a failed signup is not.
  raise warning 'handle_new_user: profile insert failed for user %: %', new.id,
sqlerrm;
  return new;
end;
$$ language plpgsql security definer;

create trigger on_auth_user_created
  after insert on auth.users
  for each row execute procedure public.handle_new_user();
```

20 Summary & Next Steps

What you now have, how to extend it, and how to maintain it

What This System Gives You

Working through this handbook gives you a complete, production-grade AI development system. Every layer is designed to work together:

`Spec Files (/ai/)`	AI always knows the architecture — no hallucinated tables or components
`Context Anchors (/ai/context/)`	Short-memory problem solved — AI stays consistent across long sessions
`Migration System (/db/migrations/)`	Database evolves safely — no silent destructive changes
`Task System (/ai/tasks/)`	Features are planned before coded — AI executes, not improvises
`Security System (/ai/security.md)`	RLS is correct by default — no dangerous open policies
`Feature Packs (/feature-packs/)`	Reusable modules — auth, teams, notifications ready in minutes
`Cross-Platform Schema (/ui-schema/)`	Design once — generate for web, iOS, Android, desktop
`Prompt Library`	25+ copy-paste prompts — one for every development situation
`Checklist + Lessons`	AI self-audits — mistakes don't repeat across sessions

Extending the System

Adding New Features

Write a task file → generate migration → build → audit → commit. Never skip the migration step.

Adding New Platforms

Create a new entry in /ui-schema/features/[feature].yaml with the platform in the platforms list, then run the platform generation prompt.

Adding New Feature Packs

Build the system in a project, then extract it: create feature.md, schema.sql, policies.sql, tasks.md, and move components to the pack folder.

Growing the Anchor System

When a subsystem grows to 5+ files, create a dedicated context anchor summarizing it. Add it to the standard preamble for that subsystem's prompts.

Onboarding Collaborators

Point them to /ai/brain.md first. The brain file is the entry point — anyone reading it understands the system within minutes.

Maintaining the System

- After every migration, update /db/schema.sql — this is the single source of truth for current database state
- After every AI mistake, add an entry to /ai/lessons.md — this file compounds in value over time
- After every significant feature, update the relevant anchors if tables or components changed
- Run the security audit prompt at least once per project milestone
- Keep feature packs updated — when you improve a system in one project, backport the improvement to the pack

✓ The Most Important Habit

Always begin every Cursor or Claude prompt with: Follow /ai/brain.md. This single habit is responsible for 80% of the reliability improvement this system provides. Without it, AI drifts from the architecture within a few exchanges.

Copyright Anders Frederik Binau, 2026-03-08

All rights reserved. This handbook is intended for personal and commercial use by the author. Reproduction or distribution without permission is prohibited.